

UNIVERSITÀ DELLA CALABRIA

DIPARTIMENTO DI MATEMATICA E INFORMATICA



CORSO DI LAUREA MAGISTRALE IN INFORMATICA

Parallelizing computation by OpenMP tailored on Mie-theory
for multiple gold nanoparticles

Relatore:

Prof. William Spataro



SPATARO
WILLIAM
05.03
.2021
15:39:58
UTC

Candidato:

Pietro Napoli

193671

Co-relatori:

Prof. Roberto Caputo

Dott. Giuseppe Emanuele Lio

ANNO ACCADEMICO 2020/2021

*Alla mia metà,
la mia ispirazione,
a colei che mi guida
e mi accompagna per mano
in questa nostra vita insieme,
a te Serena.*

...

Contents

Introduction	1
1 Mie scattering and computer science solutions	3
1.1 Complex refractive index and relative permittivity	3
1.2 Outline of Mie's theory	4
1.3 Localized Surface Plasmon Resonance (LSPR)	6
1.4 Computer science solutions	7
1.4.1 COMSOL Multiphysics	8
1.4.2 Finite Element Method (FEM)	8
1.4.3 Drawback of the COMSOL approach	9
1.5 Plasmonic Coupling	10
1.6 Our solution	16
2 Parallel Computing and OpenMP	17
2.1 Parallel Computing	17
2.1.1 Shared Memory	21
2.1.2 Distributed Memory	22
2.2 OpenMP overview	23
2.2.1 Fork-join model	24
2.2.2 Parallel construct	25
2.2.3 Worksharing constructs	27
2.2.3.1 Loop construct	27
2.2.3.2 Sections construct	28
2.2.3.3 Single construct	29
2.2.4 Synchronization constructs	30
2.2.4.1 Master construct	30
2.2.4.2 Critical construct	31
2.2.4.3 Barrier construct	32
2.2.4.4 Locks	33
2.3 Related Work	34
3 Mie scattering simulation	38
3.1 Overall System Implementation	38
3.2 Logical Structures	39

3.3	Neighborhood	41
3.4	Phases	42
3.4.1	Pattern creation phase	42
3.4.2	Constraint application phase	44
3.4.3	Simulation phase	48
3.5	Parallelization	51
4	Results and simulations	56
4.1	Physical results	58
4.2	Performance results	61
4.3	Performance evaluations	67
5	Conclusions	70
A	Simulation phase	72
B	Gmie function	83
	Bibliography	85

Introduction

In the present thesis work a physical system has been implemented that simulates the application of Mie theory on a considerable distribution of nanoparticles having a certain diameter and a certain type of material. To our knowledge, no code written in C/C++ or other modern programming languages that implemented a system designed for the same purpose is currently available. In fact, everything that was found was based on simple codes that took into account a single nanoparticle, many of which are written in programming languages that are now obsolete. As illustrated by the co-supervisors of the Department of Physics of the University of Calabria (Italy) who deal precisely with the study of this topic, there is currently no program or tool available to help them in the study of this particular theory, or at least there is no system that succeeds for computational reasons to simulate a distribution that takes into account more than a few tens of nanoparticles. In fact, most of the application programs used in the physical field are proprietary software that have not been implemented for a specific use of this type. The theory in question deals with quantifying the electromagnetic fluxes resulting from the application of an electromagnetic wave on a distribution of nanoparticles having a certain type of material. In particular, the theory models the laws that allow to quantify the diffused energy and that absorbed by the nanoparticles, when these are made of a certain material and are irradiated by a certain wavelength. The resulting energy flows are influenced both by the type of material used which, as described later in the appropriate paragraph, may be subject to losses, but also by how the nanoparticles are distributed, i.e. by the distance between them. In fact, the more the distance is reduced, the more the resonance effect resulting from the interaction between the electromagnetic wave and the material of the nanoparticles will be. In order to apply this theory it was necessary to implement a system that not only took into account the theory itself but also took care of the creation of the distributions to be studied. To this end, a system was implemented that

would allow to create a distribution of nanoparticles in a random way on which to subsequently simulate the theory in question. The part of the system dedicated to the implementation of the theory to be applied to the distribution of nanoparticles created was based on a code written by Nobuhiro Moteki present on the GitHub platform, which implements this theory by considering a single nanoparticle. This code was integrated and expanded in order to obtain a code that took into account more nanoparticles, and could consider them as a whole so as to be able to quantify the influence that the neighboring nanoparticles exert. To do this, the use of the plasmon rule equation has been integrated into the code, ie an equation defined for this purpose which will be described in detail in the following paper. The purpose of this system was immediately to be able to simulate a distribution formed by a large number of nanoparticles in an efficient way. In fact, for this purpose, once some improvements have been made to the code, it has been further expanded through the use of the OpenMP library so that its execution could be parallelized in a shared memory environment, using more processors and therefore more exploiting the computing power available in modern multi-core systems, so as to be able to simulate distributions in a shorter time or be able to simulate larger distributions efficiently. In this thesis we will describe in detail the reason for some IT choices regarding the type of representation of the nanoparticles and the data structures delegated to manage them, both from the computational point of view and from the representative point of view in memory. Choices that are the result of considerations based on the problems highlighted in other physical software that come out when they try to represent similar systems, but also to make the most of the strengths of these logical representations that have led to fairly high performance of the system itself, especially in its parallel version.

Chapter 1

Mie scattering and computer science solutions

1.1 Complex refractive index and relative permittivity

In order to explain how materials influence electromagnetic radiations and obtain consequently a plasmonic resonance, it's important to understand that each material is characterized by a refractive index that is a complex refractive index ($\tilde{n}$) and the relative permittivity (ϵ). In optics, the refractive index (also known as refraction index or index of refraction) of a material is a dimensionless number that describes how fast light travels through the material. It is defined as:

$$\tilde{n} = \frac{c}{v} \quad (1.1)$$

where c is the speed of light in vacuum and v is the phase velocity of light in the medium. For example, the refractive index of water is 1.333, meaning that light travels 1.333 times slower in water than in a vacuum. Increasing the refractive index corresponds to decreasing the speed of light in the material. The refractive index varies with wavelength, this causes white light to split into constituent colors when refracted. This is called dispersion. It can be observed in prisms and rainbows, and as chromatic aberration in lenses. Light propagation in absorbing materials can be described using a complex-valued refractive index.

The imaginary part then handles the attenuation, while the real part accounts for refraction. For most materials the refractive index changes with wavelength by several percent across the visible spectrum [1]. Instead the relative permittivity, or dielectric constant, of a material is its (absolute) permittivity expressed as a ratio relative to the vacuum permittivity [2]. In electromagnetism, the absolute permittivity ϵ , is a measure of the electric polarizability of a dielectric. A material with high permittivity polarizes more in response to an applied electric field than a material with low permittivity, thereby storing more energy in the material [3]. Permittivity is typically associated with dielectric materials, however metals are described as having an effective permittivity, with real relative permittivity equal to one. In the low-frequency region, which extends from radio frequencies to the far infrared and terahertz region, the plasma frequency of the electron gas is much greater than the electromagnetic propagation frequency, so the refractive index $\tilde{n}$ of a metal is very nearly a purely imaginary number. In the low frequency regime, the effective relative permittivity is also almost purely imaginary: It has a very large imaginary value related to the conductivity and a comparatively insignificant real-value. In our case must be take in consideration, relative permittivity for lossy materials formulated as:

$$\epsilon_r = \epsilon'_r - \frac{i\sigma}{\omega\epsilon_0} \quad (1.2)$$

in terms of a "dielectric conductivity" σ (units S/m, siemens per meter), which "sums over all the dissipative effects of the material; it may represent an actual [electrical] conductivity caused by migrating charge carriers and it may also refer to an energy loss associated with the dispersion of ϵ' [the real-valued permittivity]". Expanding the angular frequency $\omega = \frac{2\pi c}{\lambda}$ and the electric constant $\epsilon_0 = \frac{1}{\mu_0 c^2}$, where λ is the wavelength and c is the speed of light in vacuum [2].

1.2 Outline of Mie's theory

The optical response of metal nanoparticles illuminated by electromagnetic radiation can be modeled by Mie's theory. The optical properties of the nanoparticles can be visualized by the presence of a resonance peak in the absorption spectrum. The resonance modes are dependent on the size of the particles and in addition they differ by their field distribution and charge, their shape and the surrounding

environment, which is characterized by its own dielectric constant. The size of the nanoparticles must always be considered with respect to the wavelength [4] and only certain configurations allow to know the exact solution of the optical response. The one who was responsible for proposing a theory that provided an exact solution for the scattering and absorption by spherical nanoparticles was Gustav Mie, in 1908. The Mie solution to Maxwell's equations (also known as the Lorenz–Mie solution, the Lorenz–Mie–Debye solution or Mie scattering) describes the scattering of an electromagnetic plane wave by a homogeneous sphere. The solution takes the form of infinite series of spherical multipole partial waves. The term Mie theory is sometimes used for this collection of solutions and methods; it does not refer to an independent physical theory or law [5]. Maxwell's equations instead are a set of coupled partial differential equations that, together with the Lorentz force law, form the foundation of classical electromagnetism, classical optics, and electric circuits. The equations provide a mathematical model for electric, optical, and radio technologies. They describe how electric and magnetic fields are generated by charges, currents, and changes of the fields [6]. The Mie's theory is usually used to derive approximate solutions for affine geometries [7]. According to this theory, electromagnetic fields are expanded into spherical harmonics and all possible, Localized Surface Plasmon Resonance (LSPR) modes correspond to the dipolar and multipolar electromagnetic eigenvalues of the particle [8]. The scattering of the incident electromagnetic radiation of the nanoparticles is attenuated by plasmon interaction processes; in part the radiation is diffused in various directions and in part absorbed, so the energy is converted into internal energy of the nanoparticle. Processes of this type are known as absorption and diffusion. This is due to the fact that the electrons that are inside the nanoparticle, to the passage of electromagnetic radiation at a given wavelength, react to the electric field linked to the incident radiation producing a resonance phenomenon. The attenuation of the incident radiation, due to absorption and diffusion, is called extinction. Recent advances in research have allowed the study and analysis of materials at the nanometric scale, but above all to confirm the interest in the surface plasmon response of these materials in various disciplines such as physics, chemistry, materials science, biology and medicine.

1.3 Localized Surface Plasmon Resonance (LSPR)

The study of the interaction of the electromagnetic field with conduction electrons is carried out by plasmonics, which is a branch of nano-photonics that studies the interaction between light and structured materials on a nanometric scale. It stands on the mutual influence of electromagnetic fields and the conducting electron gases of metals (plasma). The materials in question are typically structured in the form of metal nanoparticles. The nanoparticles in question have a smaller diameter than the wavelength of the incident light. Conduction electrons respond instantly to the oscillation of the electric field over time, since they are weakly bound to metal ions. This effect causes a collective oscillation of the electrons, called plasmon, and they all move in phase at a certain frequency, defined as plasma frequency. The collective motion of these electrons is called plasmonic motion. The electron cloud, when these are irradiated by an electromagnetic field, undergoes a variation because the field induces a polarization of these: on their surface there is the formation of a dipole and therefore the arrangement of a negative charge on one extreme and one positive, due to ions, on the opposite extreme. The condition is shown below:

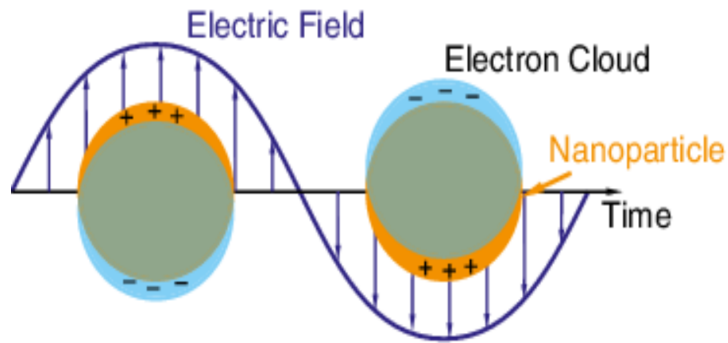


FIGURE 1.1: The light incident on a metal nanoparticle causes the electrons of the conduction band to oscillate. This is the localized surface plasmon [9].

Between the electrons and the nuclei there is an attractive force that tries to restore the initial situation and therefore to modify the electronic oscillation with respect to the general picture, this is because the redistribution of the positive and negative charges generates a recall force that brings the system back to the conditions initials. The set of electrons can be considered as a plasma that has a well-defined

plasma frequency which, if it is similar to the frequency of the incident radiation, generates a plasmon resonance phenomenon. The plasma frequency depends on several factors such as electron density, effective electronic mass, shape and size of the nanoparticles. When plasmon resonance refers to a particle that shows dimensions comparable or smaller than electromagnetic radiation, the collective excitations of the conduction electrons, due to the electromagnetic field acting from the outside, do not propagate. Such collective oscillations, in resonant conditions, are localized near the surface of the material. The resonance in question, for the reason just reported, is the so called LSPR. The absorption of light by the particle depends on it and this is usually represented in the absorption spectrum, in the form of bands. When the interaction between the incident electromagnetic field and the nanoparticle under consideration is very intense, that is, when the system is in full resonance, the intensity of the bands is higher. Following this absorption, the whole of the nanoparticles takes on a certain color, which is not given by the absorbed white light component, rather by the reflected one, which is its complement.

1.4 Computer science solutions

From Computer science point of view, there are many software solutions, but each of it has some drawbacks like those described below. From the open-source side, codes found on web shows that the most C, C++, Fortran and other similar codes writed with programming languages like these in order to apply Mie scattering solution to Maxwell's equations, they take in consideration just a nanoparticle, and usually the entire system is computed as function of a single wavelength. On the other side, proprietary software like MATLAB and COMSOL allow with some adjustment to take in consideration more then one nanoparticle, but due at the approach implemented on these system described later, they are not able to work with more then few dozen of nanoparticle. In this thesis work was taken into consideration the software COMSOL as a reference point for the overall duration of the code implementation.

1.4.1 COMSOL Multiphysics

COMSOL Multiphysics is a simulation platform that encompasses all steps of the modeling workflow: from defining geometries, material properties and physics for describing specific phenomena, to resolution and post-processing to produce accurate and reliable results [10]. Through this software it is possible to consider phenomena that involve several physical phenomena (called multiphysics) or coupled and above all with the option of being able to choose additional modules, which link the simulations and a program on technical calculation, such as CAD and ECAD. Furthermore, the platform lends itself to being usable for the study of different phenomenological situations and for the development of devices in various fields, for example in the electromagnetic, thermodynamic, mechanical, chemical and fluid dynamic fields. The effectiveness of COMSOL focuses particularly on the finite element method (FEM). The software performs the analysis by simultaneously generating the adaptive mesh and checking for errors with a variety of numerical solvers with the aim of ensuring a consistent result with very minimal relative errors.

1.4.2 Finite Element Method (FEM)

As mentioned early, COMSOL makes use of FEM methodology to describe the overall system. FEM is the most widely used method for solving problems of engineering and mathematical models. Typical problem areas of interest include the traditional fields of structural analysis, heat transfer, fluid flow, mass transport, and electromagnetic potential. The FEM is a particular numerical method for solving partial differential equations in two or three space variables (i.e., some boundary value problems). To solve a problem, the FEM subdivides a large system into smaller, simpler parts that are called finite elements. This is achieved by a particular space discretization in the space dimensions, which is implemented by the construction of a mesh of the object: the numerical domain for the solution, which has a finite number of points. The finite element method formulation of a boundary value problem finally results in a system of algebraic equations. The method approximates the unknown function over the domain. The simple equations that model these finite elements are then assembled into a larger system of

equations that models the entire problem. The FEM then uses variational methods from the calculus of variations to approximate a solution by minimizing an associated error function [11].

1.4.3 Drawback of the COMSOL approach

Although the fem approach facilitates the description and implementation of systems with truly very complex models, from the IT point of view it involves a considerable increase in the computational work required to solve them. This can be deduced simply by pausing to analyze how the entire system is described inside, that is the discretization of the space itself with the creation of the mesh necessary to describe in detail the object under analysis in its complexity. For example, taking into consideration the current thesis topic, consider the representation of a nanoparticle with the model of a sphere, this sphere is processed by the FEM methodology and is then discretized into smaller and simpler parts, until the so-called finite elements are obtained. Which in the trivial case of a sphere correspond to the single vertices, so in other words this process ends with the creation of the mesh of the sphere itself. It is easy to understand that in order to obtain greater precision in terms of results, one must necessarily have a mesh as accurate as possible and faithful to reality, and this in mathematical terms can be translated into an exponential increasing in the number of vertices of the mesh itself. All this implies from the IT point of view a relative increase of finite elements to be processed during the simulation. Taking into consideration a system with several nanoparticles, it is easy to understand why software like COMSOL struggle to manage more than a few tens of nanoparticles.

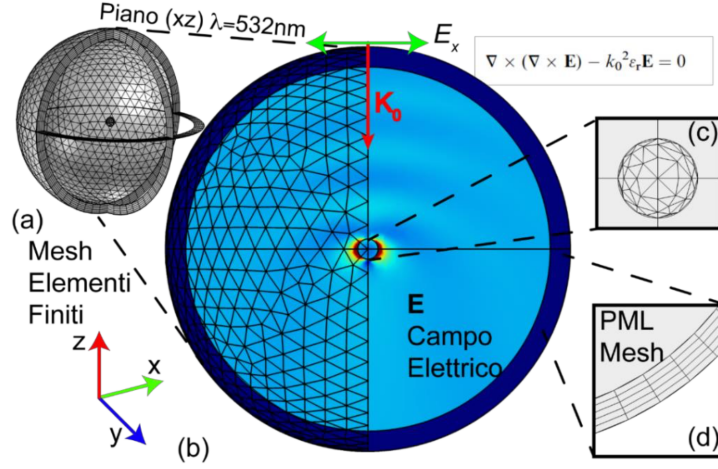


FIGURE 1.2: COMSOL model of a nano-sized sphere surrounded by a single sphere, the "perfect matched layer" (PML) or "world" of the model, made up of air and contained in a PML layer, on which a series of meshes (tetrahedral , triangular, swept) (a) and (d). Using the x, y and z axes it is possible to rotate the structure and see it from the desired point of view (b). The mesh covering the central nanoparticle is denser (c), this is due to the need to use a greater number of finite elements in order to solve the problem.

These issues have already been raised by Giuseppe Emanuele Lio and Roberto Caputo, in some of their publications in which they addressed the plasmonic issue about elongation and plasmonic coupling as reported in "Opto-mechanical control of flexible plasmonic materials"[12], "Tensile control of the thermal flow in plasmonic heaters realized on flexible substrates" [13] and "Opto-mechanically induced thermoplasmonic response of unclonable flexible tags with hotspot fingerprint" [14], in which they studied the effects of plasmonic coupling and strain stress elongation studying what happens on the side of plasmonic resonancies and thermal effects, and how to use this two effects to produce under counterfeit labels. In fact in these works the same authors underline that the study is very advantageous, but unfortunately we must take into account the limit deriving from the number of nanoparticles that can be taken into consideration by the system.

1.5 Plasmonic Coupling

Among the consequences of Localized Surface Plasmon Resonance we certainly find plasmonic coupling, that is, the phenomenon that determines the red-shift of

the resonance curves and maximum absorption at lower incident wave frequencies. Whenever two nanoparticles are placed side by side, the electric field next to the surface of one of the two inevitably interacts with that of the nearby particle. For this reason, each particle is affected by a total electric field $\overline{E}$ given by the contribution of the electric field, $\overline{E}_0$ which affects the nanoparticle, and by the electric field near the surface of the adjacent nanoparticle or near-field, $\overline{E}_{nf}$ [8]:

$$\overline{E} = \overline{E}_0 + \overline{E}_{nf} \quad (1.3)$$

The phenomenon is called plasmonic coupling because the plasmonic oscillations of the respective nanoparticles couple. The potential interaction energy, in the presence of coupled nanoparticles is lower than in the case of the isolated particle. This is valid unless the two particles are excited with fields in different directions which could prevent plasmonic coupling. The distance at which the two nanoparticles are placed determines the red-shift, while the plasmonic coupling regulates the general resonant frequency of the system. In systems that include numerous nanoparticles, each of them is affected by the near field of numerous nearby particles; coupling, in such a context, is much more intense and therefore there is a greater red-shift. Numerous factors determine the intensity of plasmonic coupling. Among these, the distance at which the nanoparticles are placed play an important role, the reason for which is the rapid decay of the field near the surface of the adjacent nanoparticle and the polarization of the incident radiation with respect to the axis that divides the two coupled nanoparticles. Considering a nanoparticle dimer in an electric field, this can be studied as a coupled dipole system [15]. Suppose the two nanoparticles come close. By going to excite with a polarization along the x axis, two dipoles are automatically produced. A condition thus presented does not show any plasmonic coupling because the nanoparticles do not interact in an ideal way:

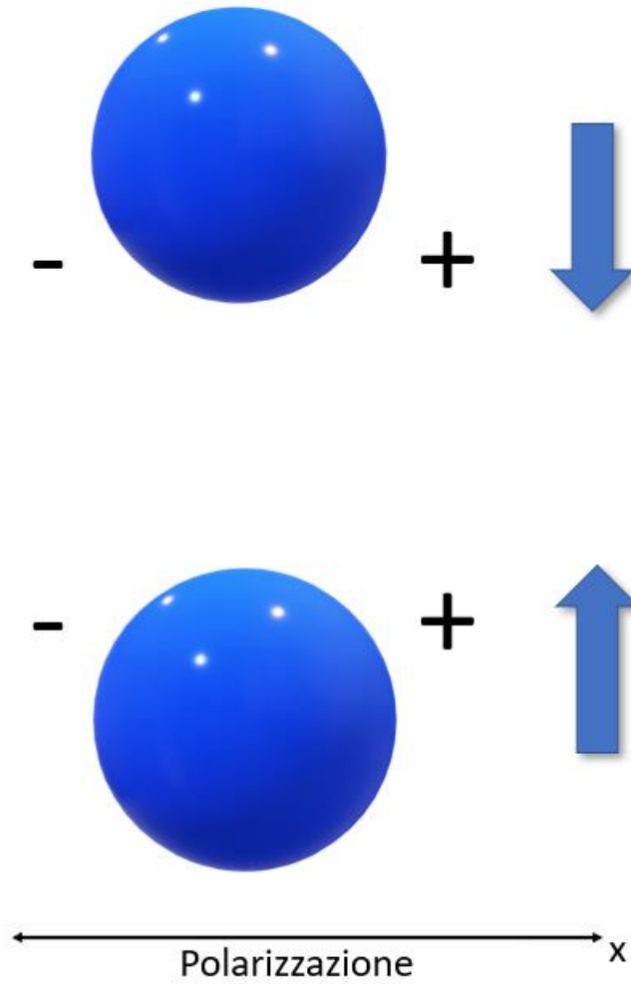


FIGURE 1.3: Two dimers that try to get closer without succeeding because plasmonic coupling for a situation of this type is not possible. The polarization is along the x axis. The blue arrows indicate that the two particles are getting closer.

In a similar situation, however, by setting the polarization along the y axis, two dipoles are obtained:

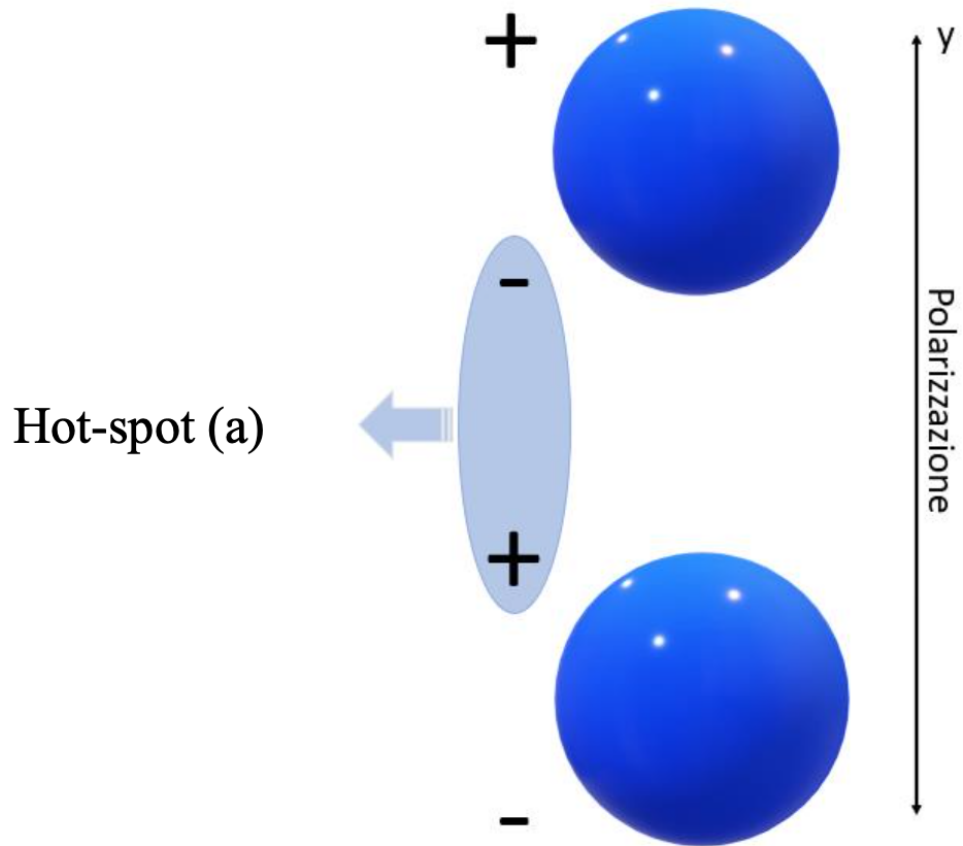


FIGURE 1.4: Two dimers that try to get closer but with polarization, in this case, along the y axis. A strong localized electric field is generated between the two nanoparticles, called hot-spot (a). For this reason, the system behaves like a single dipole.

A strong localized electric field is generated between the two nanoparticles, commonly called hot-spot. The system, therefore, behaves like a dipole whose axis is much longer than that of the dipoles linked to the nanoparticles considered individually. Therefore, it is as if there are no longer just two nanospheres, but a single nanorod, that is, particles with a tapered shape. Contrarily, for two nanoparticles that move away from each other and excited with a polarization along the x axis, the condition is shown below:

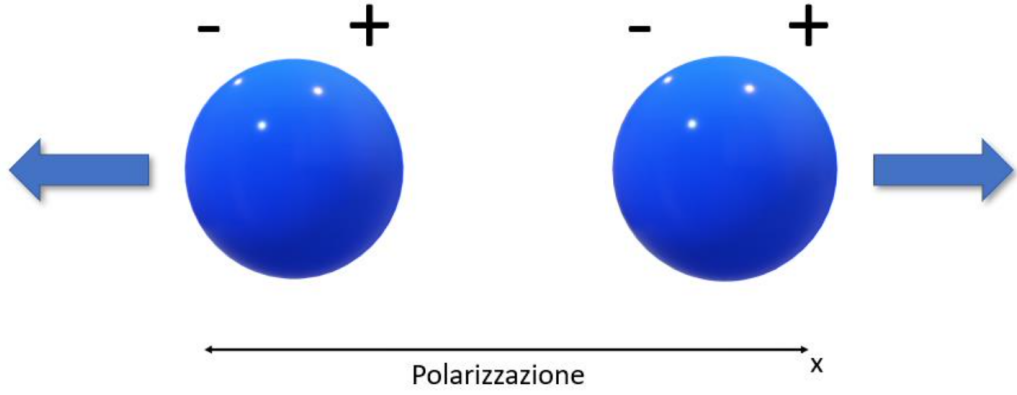


FIGURE 1.5: Two dimers that move away and have a polarization along the x axis; plasmonic coupling, for a situation of this type, occurs due to the orientation between the dipoles.

Here we have a coupling condition again, this is because the two dipoles orient themselves. The two nanoparticles, since they are removed, no longer interact in an ideal manner and, for this reason, from an electrostatic point of view, two charges that move away cause a decrease in strength and therefore also in energy. The hot spot created between the two nanoparticles is less intense and therefore a blue-shift is obtained. The nature of the plasmonic displacement due to the grouping of nanoparticles derives from the strength of the interparticle coupling, which in turn depends on the proximity of the individual nanoparticles. Therefore, the plasmonic displacement gives a quantitative evaluation of the distance between the nanoparticles. In order to measure distances at the nanoscale, the "**plasmon ruler**", ie the rule of plasmonic coupling was developed following numerous studies. The "**plasmon ruler equation**" was derived estimating the interparticle separation, starting from a plasmonic displacement observed experimentally for a pair of *Au* nanoparticles [16]:

$$\frac{\Delta\lambda}{\lambda_0} \approx 0.18e^{\left(\frac{-\left(\frac{S}{D}\right)}{0.23}\right)} \quad (1.4)$$

where $\Delta\lambda/\lambda_0$ represents the fractional plasmonic displacement, S is the edge-to-edge interparticle separation and D is the diameter of the nanoparticles. A fundamental prerequisite for the application of this law is the systematic calibration and

standardization of the displacement as a function of interparticle separation. The plasmonic coupling depends on the polarization, in fact there is an exponential decay of the plasmonic displacement due to the polarization along the interparticle axis. For the polarization parallel to the interparticle axis, the plasmon resonance undergoes a strong red-shift as the interparticle distance (gap) is reduced. On the other hand, there is a shift towards blue, therefore a very weak blue-shift, due to a decreasing gap, that is, when the polarization is orthogonal to the interparticle axis. Microabsorption spectra will be shown below for pairs of nanoparticles having different interparticle separation gaps: 212, 27, 17, 12, 7 and 2 nm. Two different polarization directions of the incident light have been chosen, one parallel to the interparticle axis (Figure left) and the other perpendicular to the axis (Figure right):

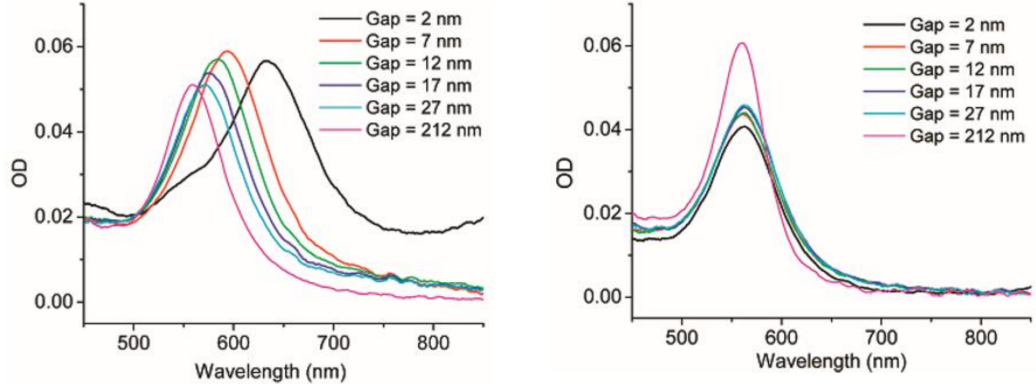


FIGURE 1.6: Microabsorption of pairs of gold nanodiscs as the interparticle separation gap varies for the direction of the polarization of incident light (left) parallel and (right) perpendicular to the interparticle axis.

The displacement of plasmon resonance derives from the electromagnetic coupling of single-particle plasmon, whose dependence on polarization can be clarified on the basis of a simple dipole-dipole coupling. For the parallel polarization the attractive dipole-dipole interaction is favored, resulting in a reduction of the plasmon frequency (red-shift), while for the orthogonal polarization the repulsive one, with the consequent increase in the plasmon frequency (blue-shift). Therefore, it is possible to deduce that the interparticle interactions are stronger due to the parallel polarization, which is observable from the shifts towards longer wavelengths.

1.6 Our solution

The IT exposed in this thesis work has been implemented with the aim of being able to easily perform simulations that take into account a large number of nanoparticles in a simple and efficient way from the computational point of view. And to achieve these objectives, everything was based on two key points, that is, the first concerns the way in which the nanoparticles are represented within the system, and the second on how to implement the physical laws of the system itself, namely say how to adapt the necessary measures in order to take into account the interactions between several nanoparticles. Regarding the representation of nanoparticles, it was chosen to represent them simply as coordinates of a point in space, so as you can easily guess the system simply takes into account a set of points, each of which represents a nanoparticle. In this paper, nanoparticles of the same diameter are taken into consideration, but it could easily be adapted to the use of nanoparticles having their own diameter. Not using a mesh approach, therefore allowed to avoid not only the problems related to the huge number of vertices that must be taken into account for each sphere, but also to avoid implementing the FEM methodology, based on the number of finite elements, thus being able consider the nanoparticles in their entirety rather than considering them through the thousands of vertices of which is formed the single. The second key point is based on this, that is the physical laws to be implemented in the system, in fact a variant of the Mie Theory solution has been implemented that takes into account the individual nanoparticles in their entirety and exploits the plasmon ruler equation to quantify the influence that neighboring nanoparticles exert on it quantifying it based on distance. In this way it is possible to take into account the influence of neighboring nanoparticles directly during the computation phase of the current nanoparticle by exploiting the plasmon ruler equation. The approach adopted and the various IT structures implemented in order to obtain a good simulation system are written in detail in chapter 3.

Chapter 2

Parallel Computing and OpenMP

2.1 Parallel Computing

Enhancing computer application performance is the only practical purpose of parallel processing. Many computer applications continue to exceed the capabilities delivered by the fastest single processors, so it is compelling to harness the aggregate capabilities of multiple processors to provide additional computational power. Even applications with adequate single-processor performance on high-end systems often enjoy a significant cost advantage when implemented in parallel on systems utilizing multiple, lower-cost, commodity microprocessors. Raw performance and price performance: these are the direct rewards of parallel processing. The cost that a software developer incurs to attain meaningful parallel performance comes in the form of additional design and programming complexity inherent in producing correct and efficient computer code for multiple processors [17]. In the more simple sense, Parallel Computing is the simultaneous use of multiple computation resources to solve a computational problem:

1. Use of multiple CPUs
2. A problem is broken into discrete parts that can be solved concurrently
3. Each part is further broken down into a series of instructions
4. The instructions of each part are performed simultaneously on different CPUs

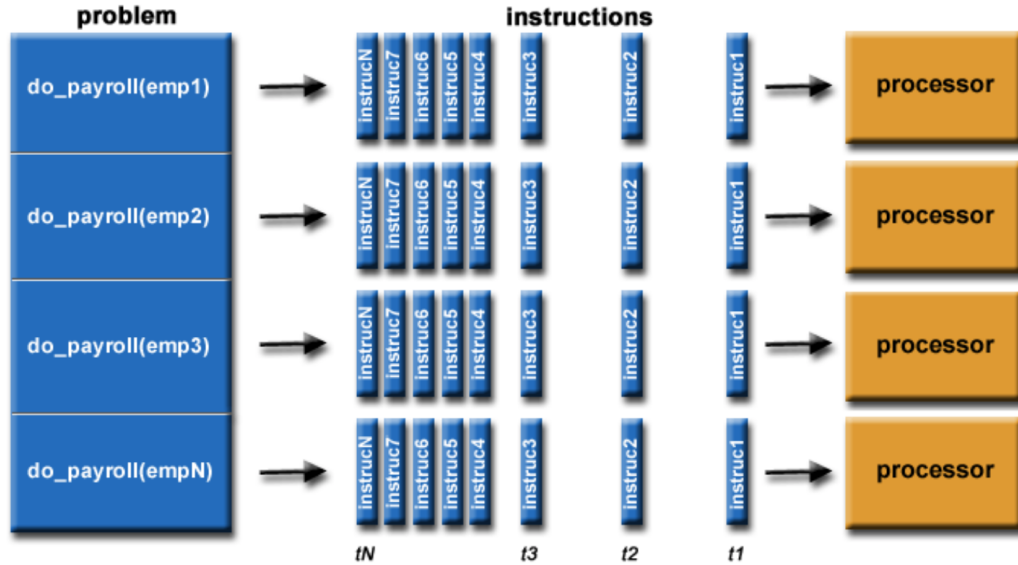


FIGURE 2.1: The way in which the problem is divided into discrete parts each of which will be further subdivided into simple instructions that will then be executed simultaneously on different CPUs.

In computing, a parallel programming model is an abstraction of parallel computer architecture, with which it is convenient to express algorithms and their composition in programs and its the value can be judged on its generality: how well a range of different problems can be expressed for a variety of different architectures, and its performance and how efficiently the compiled programs can execute. The implementation of a parallel programming model can take the form of a library invoked from a sequential language, as an extension to an existing language, or as an entirely new language. Consensus around a particular programming model is important because it leads to different parallel computers being built with support for the model, thereby facilitating portability of software. In this sense, programming models are referred to as bridging between hardware and software. Classifications of parallel programming models can be divided broadly into two areas:

- process interaction
- problem decomposition

Process interaction relates to the mechanisms by which parallel processes are able to communicate with each other. The most common forms of interaction are

shared memory and message passing (Distributed Memory), but interaction can also be implicit (invisible to the programmer)[18].

- Shared memory is an efficient means of passing data between processes. In a shared-memory model, parallel processes share a global address space that they read and write to asynchronously. Asynchronous concurrent access can lead to race conditions, and mechanisms such as locks, semaphores and monitors can be used to avoid these. Conventional multi-core processors directly support shared memory, which many parallel programming languages and libraries, are designed to exploit, one of this is OpenMP.
- In a message-passing model, parallel processes exchange data through passing messages to one another. These communications can be asynchronous, where a message can be sent before the receiver is ready, or synchronous, where the receiver must be ready, and one of the best known protocols that allows to implement the message-passing model is Message Passing Interface (MPI).
- In an implicit model, no process interaction is visible to the programmer and instead the compiler and/or runtime is responsible for performing it. Two examples of implicit parallelism are with domain-specific languages where the concurrency within high-level operations is prescribed, and with functional programming languages because the absence of side-effects allows non-dependent functions to be executed in parallel.

Problem decomposition relates to the way in which the constituent processes are formulated, that is Task parallelism, Data parallelism and Implicit parallelism.

- A Task-parallel model focuses on processes, or threads of execution. These processes will often be behaviourally distinct, which emphasises the need for communication. Task parallelism is a natural way to express message-passing communication. In Flynn's taxonomy, task parallelism is usually classified as MIMD/MPMD or MISD.
- A Data-parallel model focuses on performing operations on a data set, typically a regularly structured array. A set of tasks will operate on this data, but independently on disjoint partitions. In Flynn's taxonomy, data parallelism is usually classified as MIMD/SPMD or SIMD.

- As with implicit process interaction, an implicit model of parallelism reveals nothing to the programmer as the compiler, the runtime or the hardware is responsible. For example, in compilers, automatic parallelization is the process of converting sequential code into parallel code, and in computer architecture, superscalar execution is a mechanism whereby instruction-level parallelism is exploited to perform operations in parallel.

As we described early, there are two types of data exchange between parallel tasks:

- access to a shared data space
- through messages exchange

Platforms that provide a common space shared are called shared-address-space or Multiprocessors (shared memory) while those that allow the exchange of messages are called message passing platforms or multicomputers (distributed-memory). In the case of shared memory part (or all) the memory is directly accessible by all processors, so the processors interact by modifying data stored in this "space" shared and if the time taken by a processor to access any memory word of the system (local or global) is identical, the platform is classified as uniform memory access (UMA), otherwise, as the machine non-uniform memory access (NUMA)

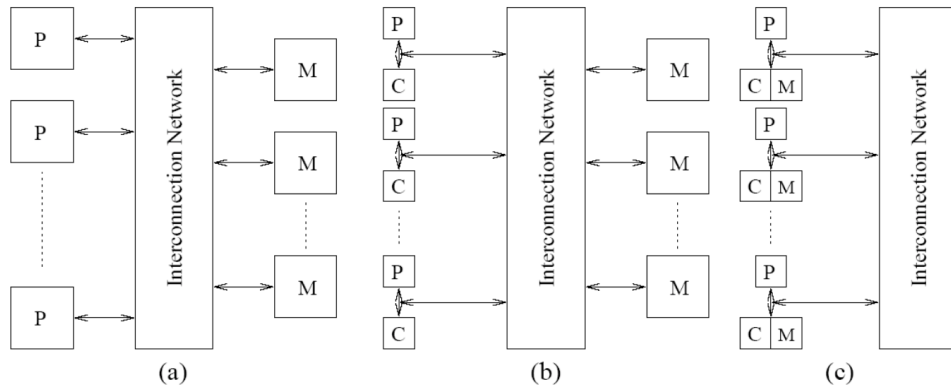


FIGURE 2.2: Typical shared-address-space architectures: (a) Uniform-memory-access; (b) Uniform-memory-access with caches, (c) Non-uniform-memory-access with local memory only.

2.1.1 Shared Memory

Shared memory systems can also vary widely but they all have the ability for each processor to access all memory as a global address space. Figure below shows an example configuration for a shared memory system where each of the processors is directly connected to the global memory. This configuration allows multiple processors to operate independently while still having access to all the same memory resources. The advantages of using a shared memory parallelization is that it is relatively simple to make existing serial codes parallel. There are a few disadvantages which include the possibility that multiple cores and processors accessing the shared memory simultaneously could cause a bottleneck which will slow down a program. Also, adding more processors does not increase the amount of memory available which could be a problem. Finally, the programmer is responsible for making sure that writing to the global memory is handled correctly. This is particularly important when a variable is being read and also written in a calculations. If multiple processors are simultaneously reading and writing, this could cause spurious results [19].

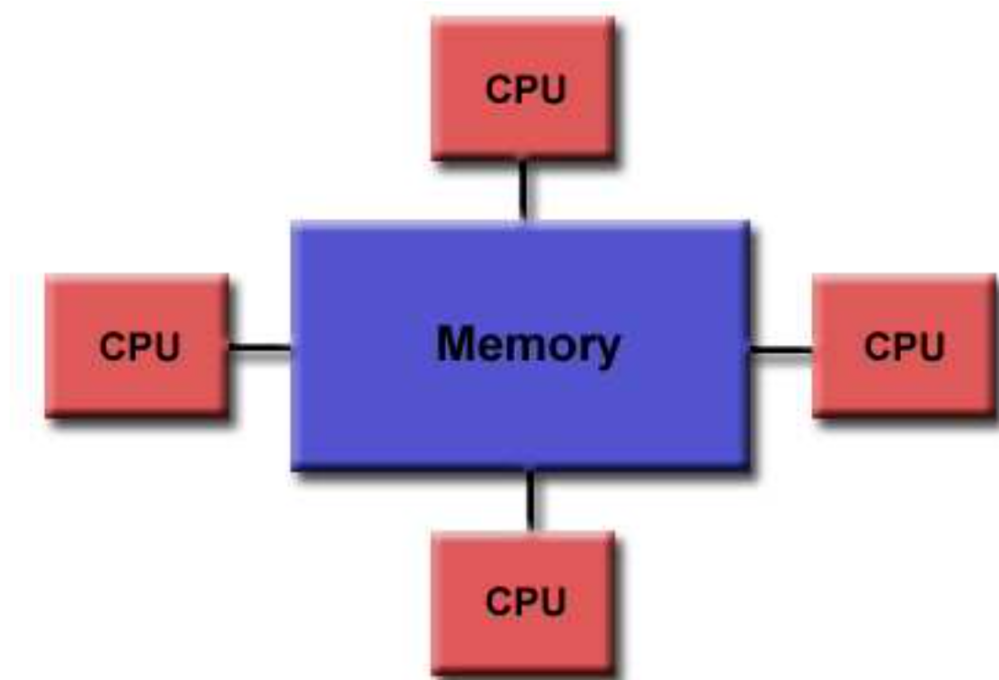


FIGURE 2.3: Typical shared memory configuration where each processor has direct access to the global memory.

2.1.2 Distributed Memory

Distributed memory systems can vary widely but they all have a communications network connecting the inter-processor memory. These network connections can be as simple as an Ethernet connection or something more sophisticated. Figure below shows an example of a distributed memory configuration with each processor having its own local memory and being connected to each other processor through some kind of network. In these configurations, memory addresses on one processor do not map to memory associated with another processor. So, since each processor has its own local memory, it operates independently of all the other processors. Consequently, when a processor is required to do a task, the programmer needs to pass all the required data to that processor explicitly. This requires specific coding in the program, which can sometimes be complicated. The advantage of using a distributed memory parallelization is that the memory is scalable. If you increase the number of processors, the size of the memory available also increases. Each processor can also access its own memory very quickly which reduces the overhead time of processors communicating with each other. The disadvantage is that it can be quite difficult to write a distributed memory parallel program and a program can not easily be made parallel after it has been written in serial without a major re-write of the code [19].

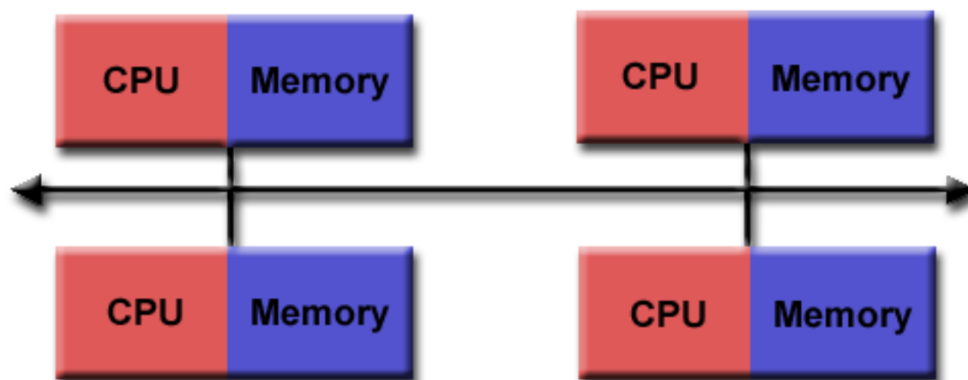


FIGURE 2.4: Typical distributed memory configuration. Each processor has its own local memory and is connected to the other processors by some kind of network.

2.2 OpenMP overview

OpenMP differs from other libraries for its features, in fact, the code written with OpenMP is portable on any platform that supports it without intervention from the programmer. The compiler deals autonomously the creation and manage threads, making life easier for the user. OpenMP aims to provide clear and straightforward directives for developing competing shared-memory applications, providing the programmer with complete control over parallelization. The aim of OpenMP is to provide a tool as transparent as possible, both user side and code side, to parallelize the execution of an algorithm designed to run in one process. To do this, the system provides a number of precompiling directives and easy-to-use, C/C++ and FORTRAN language-compatible features. OpenMP allows to indicate regions of code that can be run on parallel, in this way it is possible to incrementally convert a serial code into a parallel code without changing everything. A thread is an entity that independently runs a flow of instructions while a task is an instance of program execution. When the Operating System create a task, it allocates resources needed for the correct execution of the program, including the memory in which to load the necessary data and the program itself. In a multi-thread task, the code execution can be performed by more thread able to perform autonomy operation, but shares the resources of the parent process of which they are part. Since execution can occur independently of threads in a task, these can be run simultaneously on different processing units in a shared memory architecture. In addition, threads share most of the resources of the parent process, so creating a new thread is less burdensome than a process. OpenMP uses a multi-thread fork-join programming model. Initially in this model a task is formed from a single thread said Master. When a thread meets a parallel region, is replicated to form a team of threads. The thread that he gave life at team is the Master Thread of the team. At this point each thread can follow a flow of different instructions. Once finished the parallel operation, threads created in the fork phases join to master thread. On the execution of the parallel program can coexist many phases of fork-join. In this way the parts of code that can be parallelized can be selected. It's possible concentrate on the parts of the code that impact most performance, leaving out parts that, at the expense of a major difficulty in paralleling, would lead to irrelevant improvements or even worse. Even using the fork-join model, OpenMP hide the implementation details of low level. With OpenMP is possible to compile the same source code in order to produce

both parallel and sequential programs, in the sequential case the compiler will ignore the directives. This makes a code implemented with OpenMP usable on any platform, even those where it is not possible to run the parallel version. All this is handled transparently by the low-level compiler, with no worries for the end user [20].

2.2.1 Fork-join model

OpenMP operates on a Fork and Join model of parallel execution and this is shown in Figure below. As mentioned previously All OpenMP programs begin as a single process which is called the master thread. This master thread executes sequentially until a parallel region is encountered. At this point the master thread ‘forks’ into a number of parallel worker threads. The instructions in the parallel region are then executed by this team of worker threads. At the end of the parallel region, the threads synchronise and join to become the single master thread again. Usually you would run one thread per processor, but it is possible to run more. Parallelization with OpenMP is specified through compiler directives which are embedded in the source code [19]. Was chosen to implement the parallel version of the system with OpenMP because its approach adapts naturally to the structure of the entire system, implementing a parallel version by means of Data-parallel model based on shared memory. The main OpenMP directives will be shown in details in the next paragraphs.

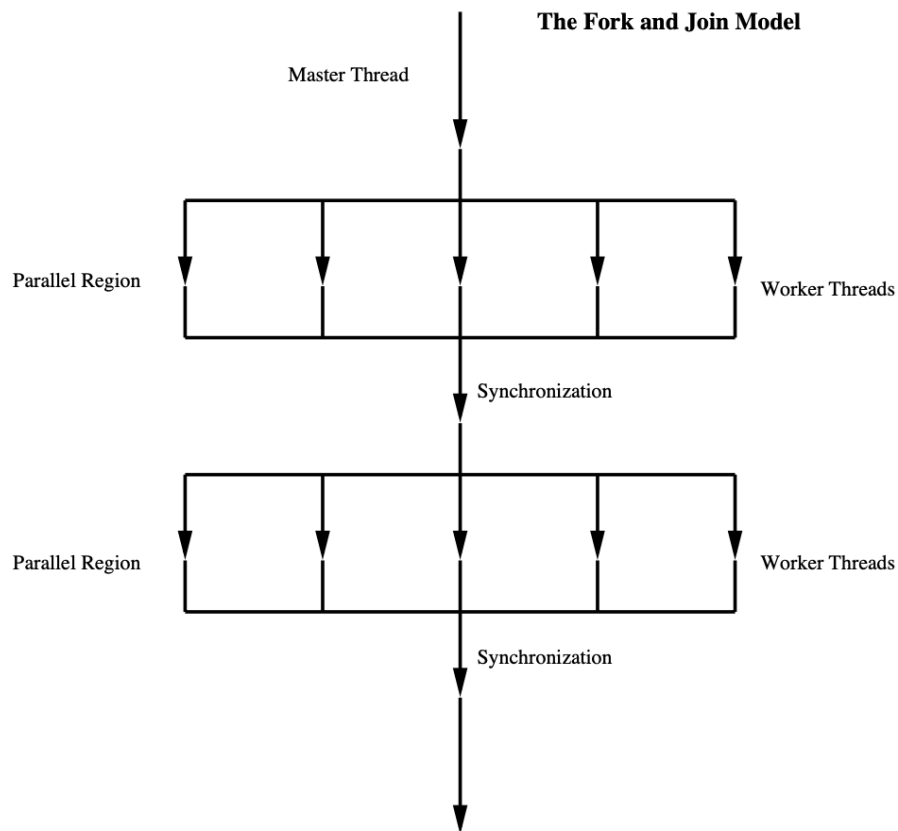


FIGURE 2.5: The fork-join model implemented by OpenMP.

2.2.2 Parallel construct

The parallel construct is the principal and simplest construct of OpenMP, it allows to define a block of parallel code, code instructions enclosed in this construct will be running on parallel, according to fork-join model previously described. When a thread encounters a parallel construct, a team of threads is created to execute the parallel region. The thread that encountered the parallel construct becomes the master thread of the new team, with a thread number of zero for the duration of the new parallel region. All threads in the new team, including the master thread, execute the region. Once the team is created, the number of threads in the team remains constant for the duration of that parallel region. Each thread of a team is assigned an implicit task consisting of code within the parallel region. The task that creates a parallel region is suspended while the tasks of the team

are executed. A thread is tied to its task; that is, only the thread assigned to the task can execute that task. After completion of the parallel region, the master thread resumes execution of the generating task. Any task within a parallel region is allowed to encounter another parallel region to form a nested parallel region. The parallelism of a nested parallel region (whether it forks additional threads, or is executed serially by the encountering task) can be controlled by the `OMP_NESTED` environment variable or the `omp_set_nested()` API routine with arguments indicating true or false. A team may be comprised of just a single thread. The number of threads of a parallel region can be set by the `OMP_NUM_THREADS` environment variable, the `omp_set_num_threads()` routine, or on the parallel directive with the `num_threads` clause. The routine overrides the environment variable, and the clause overrides all. OpenMP implementation dynamically adjust the number of threads for parallel regions. The default setting for dynamic adjustment is implementation defined. When dynamic adjustment is on and the number of threads is specified, the number of threads becomes an upper limit for the number of threads to be provided by the OpenMP runtime. Team threads will have all the variables shared by default, which means that for two different threads a generic `x` variable will have the same memory location. Sometimes, however, some variables are private, that is, each thread has a local copy. One way to define which variables should be shared and which private is to use the `private` clause followed by the list of variables we want to privatize. The value of these variables at the beginning and the exit of the region performed in parallel is undefined. If you need to initialize a variable to the value above the parallel region you have to use the `firstprivate` clause instead of `private` and, likewise, if its value should not be lost to the destruction of the team you should use `lastprivate` [20] .

Example:

```
#pragma omp parallel
{
code here will be running on parallel
}
```

2.2.3 Worksharing constructs

2.2.3.1 Loop construct

The loop construct specifies that the iterations of one or more associated loops will be executed in parallel by threads in the team in the context of their implicit tasks. The iterations are distributed across threads that already exist in the team executing the parallel region to which the loop region binds. An implicit barrier is present at end of loop region, unless a `nowait` clause is present. The parallel construct and the loop construct can be combined in `#pragma omp parallel for`, the effect is the same. The `schedule` clause can be used to determine how loop iterations are assigned to threads, if `schedule` clause is not specified, default schedule is used. The kind of schedule on OpenMP are `static`, `dynamic`, `guided`, `auto`, `runtime`. In some case the `schedule` clause accept a `chunk_size` in order to divide the iterations into chunks of size `chunk_size`. The `for` and `do` constructs (loop constructs) create a region consisting of a loop. A loop controlled by a loop construct is called an associated loop. Nested loops can form a single region when the `collapse` clause (with an integer argument) designates the number of associated loops to be executed in parallel, by forming a "single iteration space" for the specified number of nested loops. The `ordered` clause can also control multiple associated loops. An associated loop must adhere to a "canonical form" (specified in the Canonical Loop Form of the OpenMP Specifications document) which allows the iteration count (of all associated loops) to be computed before the (outermost) loop is executed. Most common loops comply with the canonical form, including C++ iterators [20].

Example:

```
int var1[N], var2[N], var3[N];
int chunk_size=M;
initVariables(var1,var2,var3);
#pragma omp parallel
#pragma omp for schedule(static,chunk_size)
for(int i=0;i<N;i++){
    var2[i]=var1[i]+var3[i];
}
```

```
}
```

2.2.3.2 Sections construct

The sections construct is a non-iterative worksharing construct that contains a set of structured blocks that are to be distributed among and executed by the threads in a team. Each structured block is executed once by one of the threads in the team in the context of its implicit task. A sections region binds to the innermost enclosing parallel region. Only the threads of the team executing the binding parallel region participate in the execution of the structured blocks. It consists of two directives: first, `#pragma omp sections` to indicate the start of the construct, and second, the `#pragma omp section` directive, to mark each distinct section except possibly the first block, for which a preceding section directive is optional. Each section must be a structured block of code that is independent of the other sections. At run time, the specified code blocks are executed by the threads in the team. The method of scheduling the structured blocks among the threads in the team is implementation defined. There is an implicit barrier at the end of a sections construct unless a `nowait` clause is specified. Obviously, if the number of sections is n , at most n threads will be engaged in executing the code. For this is important to balance as far as possible the computational load between the various sections (load-balancing). Also in this case, the abbreviated form `#pragma omp parallel sections` can be used. The sections construct is the easiest way to get different threads to carry out different kinds of work, since it permits us to specify several different code regions, each of which will be executed by one of the threads. Each thread executes one code block at a time, and each code block will be executed exactly once. If there are fewer threads than code blocks, some or all of the threads execute multiple code blocks. If there are fewer code blocks than threads, the remaining threads will be idle. Note that the assignment of code blocks to threads is implementation-dependent [20].

Example:

```
#pragma omp parallel
{
    #pragma omp sections
```

```

    {
        #pragma omp section
        funcA();
        #pragma omp section
        funcB();
    }
}

```

2.2.3.3 Single construct

The `single` construct specifies that the associated structured block is executed by only one of the threads in the team (not necessarily the master thread), in the context of its implicit task. The other threads in the team, which do not execute the block, wait at an implicit barrier at the end of the single construct unless a `nowait` clause is specified. The binding thread set for a single region is the current team. A single region binds to the innermost enclosing parallel region. Only the threads of the team executing the binding parallel region participate in the execution of the structured block and the implied barrier of the single region. The method of choosing a thread to execute the structured block is implementation defined. There is an implicit barrier at the end of the single construct unless a `nowait` clause is specified [20].

Example:

```

#pragma omp parallel shared(a,b) private(i)
{
    #pragma omp single
    {
        a = 10;
        int id=omp_get_thread_num();
        printf("Single construct executed by thread %d\n",id);
    }
// A barrier is automatically inserted here
    #pragma omp for
    for (i=0; i<n; i++)

```

```

        b[i] = a;
    }

```

2.2.4 Synchronization constructs

2.2.4.1 Master construct

The `master` construct specifies a structured block that is executed by the master thread of the team. The binding thread set for a master region is the current team. A master region binds to the innermost enclosing parallel region. Only the master thread of the team executing the binding parallel region participates in the execution of the structured block of the master region. Other threads in the team do not execute the associated structured block. There is no implied barrier either on entry to, or exit from, the master construct [20].

Example:

```

void masterExample(float* x, float* xold, int n, float tol)
{
    int c, i, toobig;
    float error, y;
    c = 0;
    #pragma omp parallel
    {
        do{
            #pragma omp for private(i)
            for( i = 1; i < n-1; ++i )
                xold[i] = x[i];

            #pragma omp single
                toobig = 0;

            #pragma omp for private(i,y,error) reduction(+:toobig)
            for(i=1; i<n-1;++i){
                y = x[i];

```

```

        x[i] = average( xold[i-1], x[i], xold[i+1] );
        error = y - x[i];
        if( error > tol || error < -tol )
            ++toobig;
    }
#pragma omp master
    {
        ++c;
        printf( "iteration %d, toobig=%d\n", c, toobig );
    }
} while( toobig > 0 );
}
}

```

2.2.4.2 Critical construct

The **critical** construct restricts execution of the associated structured block to a single thread at a time. The binding thread set for a critical region is all threads in the contention group. The region is executed as if only a single thread at a time among all threads in the contention group is entering the region for execution, without regard to the team(s) to which the threads belong. An optional name may be used to identify the critical construct. All critical constructs without a name are considered to have the same unspecified name. The threads of a contention group execute the critical region as if only one thread of the contention group is executing the critical region at a time. The critical construct enforces these execution semantics with respect to all critical constructs with the same name in all threads in the contention group, not just those threads in the current team. The presence of a **hint** clause does not affect the isolation guarantees provided by the **critical** construct. If no **hint** clause is specified, the effect is as if `hint(omp_lock_hint_none)` had been specified [20].

Example:

```

void criticalExample(float *x, float *y)
{

```

```

int ix_next, iy_next;
#pragma omp parallel shared(x, y) private(ix_next, iy_next)
{
    #pragma omp critical (xaxis)
        ix_next = dequeue(x);
    work(ix_next, x);
    #pragma omp critical (yaxis)
        iy_next = dequeue(y);
    work(iy_next, y);
}
}

```

2.2.4.3 Barrier construct

The **barrier** construct specifies an explicit barrier at the point at which the construct appears. The barrier construct is a stand-alone directive. The binding thread set for a barrier region is the current team. A barrier region binds to the innermost enclosing parallel region. All threads of the team executing the binding parallel region must execute the barrier region and complete execution of all explicit tasks bound to this parallel region before any are allowed to continue execution beyond the barrier [20].

Example

```

#pragma omp parallel
{
    work1();
    #pragma omp barrier
    work2();
}

```

2.2.4.4 Locks

The synchronization constructs seen so far are enough to solve all synchronization problems in most cases. Sometimes the programmer needs more control and flexibility. OpenMP offers, through some features of libraries and data types, the ability to use locks. The OpenMP runtime library provides a lock type, `omp_lock_t`. The principal lock type has five manipulator functions:

1. `omp_init_lock` initializes the lock. After the call, the lock is unset.
2. `omp_destroy_lock` destroys the lock. The lock must be unset before this call.
3. `omp_set_lock` attempts to set the lock. If the lock is already set by another thread, it will wait until the lock is no longer set, and then sets it.
4. `omp_unset_lock` unsets the lock. It should only be called by the same thread that set the lock; the consequences of doing otherwise are undefined.
5. `omp_test_lock` attempts to set the lock. If the lock is already set by another thread, it returns 0; if it managed to set the lock, it returns 1.

A lock must first be initialized using the function `omp_init_lock`. To acquire a lock, instead, you use the `omp_set_lock` function and release the `omp_unset_lock`. When a thread tries to acquire a lock that is not available with the `omp_set_lock` function, its execution is suspended until the lock is released (`omp_set_lock` is defined as blocker). The `omp_test_lock` function is also used to capture a thread lock. In fact, when the lock is unavailable, the function immediately returns a value other than zero, without blocking the execution of the thread. This is useful to handle synchronization even more flexibly and use for some other purpose a thread that should still wait for lock availability [20].

Example

```
int main()
{
    omp_init_lock(&my_lock);
    #pragma omp parallel
```

```

{
    int i, j;
    for (i = 0; i < 5; ++i) {
        omp_set_lock(&my_lock);
        if (i % 3)
            Test();
        omp_unset_lock(&my_lock);
    }
}
omp_destroy_lock(&my_lock);
}

```

2.3 Related Work

As mentioned previously, OpenMP allows to create a parallel program from serial native code in an incremental way, transforming more expensive serial computational parts, into parallel simply by adding precompilation directives, thus improving program performance where possible. Moreover, the same code can still be executed in serial where there are no essential requirement for a parallelization of the program. The compiler takes care of all this at a low level, this makes OpenMP easily usable within certain limits by anyone, without the need to possess specific skills in parallel computing. For this OpenMP has been highly appreciated by the scientific community in general, in fact realizations of other simulation systems, similar to this one object of study, have been developed. Some of these make use of hybrid OpenMP/MPI systems in order to exploit multiple shared memory machines together, each of which uses OpenMP and interacts with the others with MPI, thus creating a larger distributed memory system, such as a cluster. In this thesis, the approach in OpenMP to solve the plasmonic coupling issue for more than one thousand nanoparticles in a new way is addressed. Nevertheless, there are other examples of simulation systems implemented with OpenMP, such as in:

- **OpenMP parallelism for fluid and fluid-particulate systems** [21].
- **Optimization of Condensed Matter Physics Application with OpenMP Tasking Model** [22].

- **Efficient parallel CFD-DEM simulations using OpenMP** [23].
- **Gyrokinetic Semi-lagrangian Parallel Simulation Using a Hybrid OpenMP/MPI Programming** [24].

each of which is described in details below:

- **OpenMP parallelism for fluid and fluid-particulate systems** [21]: In order to exploit the flexibility of OpenMP in parallelizing large scale multi-physics applications where different modes of parallelism are needed for efficient computation, it is first necessary to be able to scale OpenMP codes as well as MPI on large core counts. In this research was implemented fine grained OpenMP parallelism for a large Computational fluid dynamics (CFD) code GenIDLEST and investigated the performance from 1 to 256 cores using a variety of performance optimization and measurement tools. It is shown through weak and strong scaling studies that OpenMP performance can be made to match that of MPI on the SGI Altix systems for up to 256 cores. Data placement and locality were established to be key components in obtaining good scalability with OpenMP. It is also shown that a hybrid implementation on a dual core system gives the same performance as standalone MPI or OpenMP. Finally, it is shown that in irregular multi-physics applications which do not adhere solely to the SPMD (Single Process, Multiple Data) mode of computation, as encountered in tightly coupled fluid-particulate systems, the flexibility of OpenMP can have a big performance advantage over MPI
- **Optimization of Condensed Matter Physics Application with OpenMP Tasking Model** [22]: The Density Matrix Renormalization Group (DMRG++) is a condensed matter physics application used to study superconductivity properties of materials. It's main computations consist of calculating hamiltonian matrix which requires sparse matrix-vector multiplications. This paper presents task-based parallelization and optimization strategies of the Hamiltonian algorithm. The algorithm is implemented as a mini-application in C++ and parallelized with OpenMP. The optimization leverages tasking features, such as dependencies or priorities included in the OpenMP standard 4.5. The code refactoring targets performance as much as programmability. The optimized version achieves a speedup of 8.0x with 8 threads and 20.5x

with 40 threads on a Power9 computing node while reducing the memory consumption to 90 MB with respect to the original code, by adding less than ten OpenMP directives.

- **Efficient parallel CFD-DEM simulations using OpenMP [23]:** The paper describes parallelization strategies for the Discrete Element Method (DEM) used for simulating dense particulate systems coupled to Computational Fluid Dynamics (CFD). While the field equations of CFD are best parallelized by spatial domain decomposition techniques, the N-body particulate phase is best parallelized over the number of particles. When the two are coupled together, both modes are needed for efficient parallelization. It is shown that under these requirements, OpenMP thread based parallelization has advantages over MPI processes. Two representative examples, fairly typical of dense fluid-particulate systems are investigated, including the validation of the DEM-CFD and thermal-DEM implementation with experiments. Fluidized bed calculations are performed on beds with uniform particle loading, parallelized with MPI and OpenMP. It is shown that as the number of processing cores and the number of particles increase, the communication overhead of building ghost particle lists at processor boundaries dominates time to solution, and OpenMP which does not require this step is about twice as fast as MPI. In rotary kiln heat transfer calculations, which are characterized by spatially non-uniform particle distributions, the low overhead of switching the parallelization mode in OpenMP eliminates the load imbalances, but introduces increased overheads in fetching non-local data. In spite of this, it is shown that OpenMP is between 50–90% faster than MPI.
- **Gyrokinetic Semi-lagrangian Parallel Simulation Using a Hybrid OpenMP/MPI Programming [24]:** This paper describes a parallel implementation of a numerical solver for the Vlasov equation. The solver is based on a kinetic model describing the motion of charged particles in a plasma. The evolution of the distribution of particles in phase space is computed with an explicit method, and we take into account the self-consistent electric field through the coupling with a Poisson type equation. This paper focus on a recently developed 5D parallel numerical application dedicated to gyrokinetic simulation of tokamak systems and ITG turbulence simulation.

A semi-Lagrangian Vlasov solver is used. A specific cubic spline interpolation allows us to formulate a domain decomposition method. A hybrid MPI/OpenMP paradigm was used to benefit from a large number of processors while reducing communication costs.

Chapter 3

Mie scattering simulation

3.1 Overall System Implementation

The code that was implemented is an evolution of a basic code written in C++ concerning the Mie Theory written by Nobuhiro Moteki and released under a free license on GitHub [25]. Initially, this code took into consideration a single nanoparticle and a single wavelength, on which the system then based the entire simulation. For the current thesis work, this code was taken as input and it was modified and appropriately expanded in order to obtain a simulation system that would take into account many nanoparticles and could simulate the entire resonance spectrum. In short, the system must set the size of the radius of the nanoparticles and the size of the two-dimensional area to be simulated, and the latter will be filled with nanoparticles of this size in a random manner. Therefore the different distributions of nanoparticles will be completely random distributions, without the need to modify everything manually, also because a manual approach would not be feasible considering a distribution of thousands of nanoparticles. The code has been coupled by another program implemented in C++ which makes use of the OpenGL library to graphically display the various distributions of nanoparticles that are generated, thus offering graphic support to the system, and thus having an idea of what will be simulated. This part is only a support and does not fall within the current thesis topic, therefore it will not be mentioned in the description of the system itself. The code in question, on the other hand, was finally expanded further with the addition of the use of the OpenMP library, easily transforming the code from its serial to parallel version. Therefore, the whole

system could run in parallel on multiple processors on shared memory architectures and make the most of the available hardware by shortening the simulation times required or by simulating systems with a greater number of nanoparticles in the same amount of time. As previously described in the paragraph about Mie's Theory, the scattering of the incident electromagnetic radiation by nanoparticles is attenuated by plasmon interaction processes; in part the radiation is diffused in various directions and in part absorbed, so the energy is converted into internal energy of the nanoparticle. Processes of this type are known as absorption and diffusion. This is due to the fact that the electrons that are inside the nanoparticle, the passage of radiation electromagnetic at a given wavelength, react to the electric field linked to the incident radiation producing a resonance phenomenon. The attenuation of the incident radiation, due to absorption and diffusion, is called extinction. This system aims to calculate precisely these results, in order to obtain as a result the entire resonance spectrum, for each input wavelength the system returns the respective amounts of absorption, scattering, and the sum of both, that is extinction, respectively defined in the code as the variables `Qabs`, `Qsca`, `Qext`.

3.2 Logical Structures

To obtain excellent results in terms of performance, both from the point of view of timings and in terms of the system's ability to simulate a huge number of nanoparticles - as has already been discussed above - and given that the problems are related to a mesh approach, it was preferred to represent the nanoparticles simply as coordinates of a point. As a consequence, the question that arose from this choice from a programming point of view is: "Where will these coordinates be stored in order to be able to access them efficiently?". And more importantly, analyzing the way in which we need to access the coordinates of a nanoparticle, but above all, given a nanoparticle how to obtain all the nanoparticles that influence the latter during the calculation process? To meet the system's calculation needs and the experience gained in the years of study, it was decided to base everything on a cell approach. A cell system is nothing more than a system in which space is divided into a finite number of cells, obtaining a logical matrix subdivision of the space. But unlike a matrix in which two indices are needed to identify the single element, in the cell approach it can be identified by means of a key, this key is

easily derived from two indices, in the case in question, these two indices coincide with the bi-dimensional coordinates of the nanoparticle. The calculations necessary to obtain the key and identify the cell to which each nanoparticle belongs are shown below:

```
int keyX = x / COL_DIVIDER;
int keyY = y / ROW_DIVIDER;
int key = keyX + keyY * ROWS;
```

where `x` and `y` are the coordinates of the nanoparticle, `COL_DIVIDER` is the width dimension of the cell, obtained from the width dimension of the simulation space divided by the number of columns. `ROW_DIVIDER` is the height dimension of the cell, obtained from the height dimension of the simulation space divided by the number of rows, while `ROWS` is the number of lines. The system currently makes use of 10 rows and 10 columns, for a total of 100 cells, identified by the value of the key which varies from 0 to 99. Let's see from the code the calculation of the combined intermediate keys to obtain the final `key`. This `key` is used to identify the cell to which the nanoparticle belongs. Each nanoparticle is stored in the form of a vector containing the `x`, `y` and `z` coordinates. The latter is not used, as the system does not simulate three-dimensional spaces for the moment. The cells in turn are based on vector structures, more specifically, each is a vector of vectors, in this way each cell can contain all the nanoparticles that fall by coordinates in the area covered by the cell itself. Finally the cells are inserted into a `<key, value>` map, in which the value is the cell, i.e. a vector of vectors and the `key` is the cell identifier, i.e. the key. Map and object vectors were chosen as structures to be used, precisely to better manage the dynamic nature of the system, which can vary based on:

- number of nanoparticles related to the randomly generated pattern
- number of cells used to divide the space
- size of the space itself

In this way it will be the data structures themselves that adapt best, resizing to accommodate more cells, or enlarge the cells themselves to be able to keep track of a greater number of nanoparticles.

3.3 Neighborhood

The concept of cells was not implemented only in order to define a data structure that contains a certain number of nanoparticles. It was chosen to introduce the concept of neighborhood between the various nanoparticles. This allows an enormous optimization of the calculation phase, since, given a nanoparticle, the possible neighboring nanoparticles to be considered during the calculation phase will be exclusively those of the same cell and those belonging to the neighboring cells. It is easy to deduce that the search for possible influences by other nanoparticles will be limited to a certain area, rather than carrying out a useless search on all space. The type of neighborhood used is a variant of the Margolus neighborhood, specifically the following neighborhood was chosen for the calculation phase in order to add the quantities deriving from the interactions between nanoparticles belonging to different cells once:

...	...	...	...	...
...	KEY + COLUMNS - 1	KEY + COLUMNS	KEY + COLUMNS + 1	...
...	...	KEY	KEY + 1	...
...	...	...	...	...

FIGURE 3.1: The neighbors of the corresponding KEY cell are the neighbor on the right, above, above right and above left.

In this way all the space will be computed starting from the first cell corresponding to the key value 0 up to the final cell at once, thus avoiding repetitions of the same comparisons between two cells, which would happen if the other neighbors were also considered.

3.4 Phases

As already briefly mentioned at the beginning of chapter 3, the implemented code is not based solely and exclusively on the simulation part, but in it we also find other phases necessary for the creation of the system in its entirety, such as the phase dedicated to the generation of the nanoparticle distribution in space, the phase of applying constraints in order to modify the pattern resulting from the previous phase and model it according to our purposes, obtaining a distribution of nanoparticles that is as close as possible to a real distribution. The implementation details of the various objects used, such as the structures of the nanoparticles and of the various cells, are described in the paragraph dedicated to logical structures. As for the implementation approaches of these phases, they were chosen with the aim of fully exploiting the strengths of the logical structures used from the IT point of view. So that it could be hoped to obtain already at the beginning excellent performances in the simulation of a system like this, in which it must be remembered that at the moment it deals with the subject only from the electromagnetic point of view, but it could be further expanded, considering everything also from a thermal point of view, thus integrating other functions and becoming even more complex.

3.4.1 Pattern creation phase

The first phase to be described concerns the creation of patterns, that is the distribution of the nanoparticles in space, which as mentioned is carried out through the choice of coordinates in a random way.

```
for (int i = start; i < total; i++){
    for (int j = start; j < total; j++){
        int randX = rand_r(&seed) % RANDOM_FACTOR;
        int randY = rand_r(&seed) % RANDOM_FACTOR;
        double x = radius + randX * RATE_RANDOM_FACTOR * end;
        double y = radius + randY * RATE_RANDOM_FACTOR * end;
        double z = 0.0;
        int keyX = x / COL_DIVIDER;
        int keyY = y / ROW_DIVIDER;
        int key = keyX + keyY * ROWS;
```

```

        vector<double> v = vector<double>();
        v.push_back(x);
        v.push_back(y);
        v.push_back(z);
        cells[key].push_back(v);
    }
}

```

The code reported describes this phase, which generates a simple random distribution of nanoparticles with two simple cycles, both performed in an interval from start to total, where start coincides with 0 and total is equal to the result of the division between the dimension of the area, that is the `end` variable and the dimension chosen for the radius of the nanoparticle, i.e. the `radius` variable. In this way, a sufficient number of nanoparticles is obtained to overfill the entire area. Two random values `randX` and `randY` are generated to be used to generate the resulting coordinates of the nanoparticle, respectively obtain the point $\langle x, y \rangle$, which will in turn be used to derive the key of the cell that will contain the nanoparticle. The result of this first phase can be seen in the following figure:

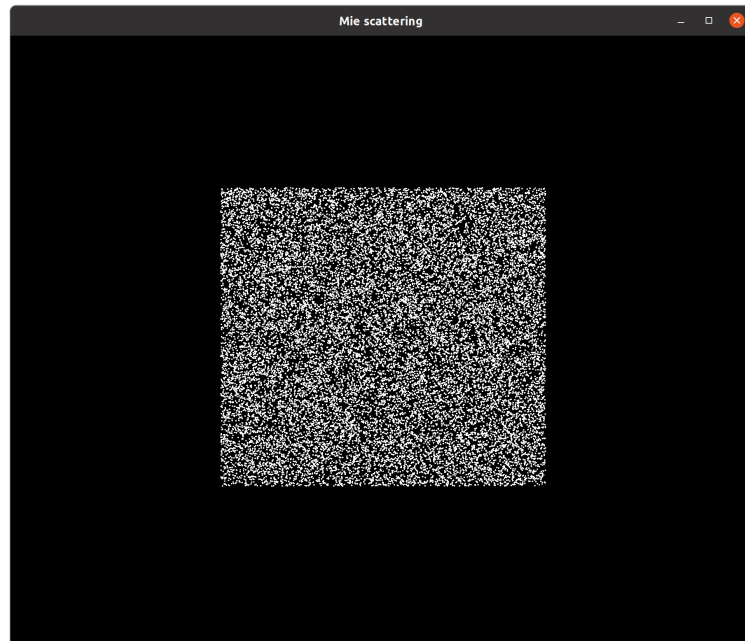


FIGURE 3.2: How is the distribution of nanoparticles for an area of $3\mu m^2$ and a total of 22500 nanoparticles, after its creation.

From the logical point of view of the cell structure instead, this distribution is divided as follows:

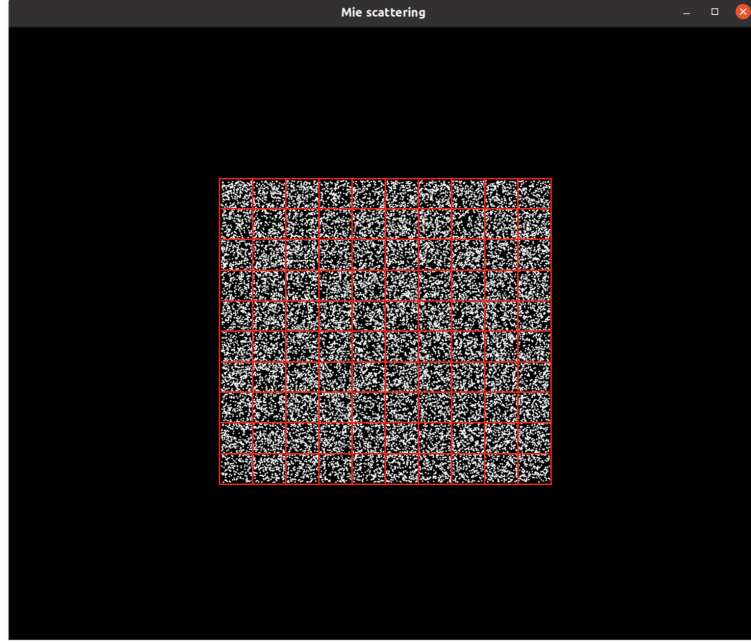


FIGURE 3.3: How is the distribution of nanoparticles for an area of $3\mu m^2$ and a total of 22.500 nanoparticles, divided by the cell structure.

3.4.2 Constraint application phase

The second phase deals with taking the generated distribution as input and modifying it in order to obtain, as already mentioned, a distribution that is as similar to a real distribution as possible. This is done through the application of constraints which will eliminate all the nanoparticles that will not respect them. In the current work only constraints on the distance between nanoparticles have been used, but others can be added at this stage of the program to obtain different distributions to study. As for cell comparison operations, Moore's neighborhood is used in this phase:

...	...	...	...	...
...	KEY + COLUMNS - 1	KEY + COLUMNS	KEY + COLUMNS + 1	...
...	KEY - 1	KEY	KEY + 1	...
...	KEY - COLUMNS - 1	KEY - COLUMNS	KEY - COLUMNS + 1	...
...	...	...	...	...

FIGURE 3.4: The neighbors of the KEY cell are all those around the cell itself.

Below is the code relating to the application of constraints:

```
void Utils::applyConstraints(){
for (map<int, vector<vector<double>>>::iterator it = cells.begin();
    it != cells.end();it++)
    withinCell(it->second);

for (map<int, vector<vector<double>>>::iterator it = cells.begin();
    it != cells.end();it++)
    withNeighbors(it->first);
}
```

The application of the constraints was divided into two steps, first it is carried out by performing comparisons between the nanoparticles within the cell itself, and then for each cell, the remaining nanoparticles are compared with the nanoparticles of neighboring cells, according to the distribution of the Moore's neighborhood.

Below is the code relating to the application of constraints between nanoparticles of the same cell:

```
void Utils::withinCell(vector<vector<double>> &cell){
for (vector<vector<double>>::iterator it = cell.begin();
      it != cell.end(); it++){
    vector<vector<double>>::iterator it2 = it + 1;
    while (it2 != cell.end()){
        if (
            checkDistanceConstraints(
                (*it)[0], //x1
                (*it2)[0], //x2
                (*it)[1], //y1
                (*it2)[1]) //y2
            )
            it2 = cell.erase(it2);
        else
            it2++;
    }
}
}
```

Below is the code relating to the application of constraints between nanoparticles of neighboring cells:

```
void Utils::withNeighbors(int key){
int numNeighbors;
int neighbors[NEIGHBORS];
getNeighbors(key, neighbors, numNeighbors);
vector<vector<double>> *current_cell = &cells[key];

for (int i_neighbor = 0; i_neighbor < numNeighbors; i_neighbor++){
    vector<vector<double>> *neighbor = &cells[neighbors[i_neighbor]];
    for (int j_sphere = 0; j_sphere < neighbor->size(); j_sphere++)
        for (vector<vector<double>>::iterator current_sphere = current_cell->begin();
              current_sphere != current_cell->end();){
```

```

    if (
    checkDistanceConstraints(
        (*current_sphere)[0], //x1
        (*neighbor)[j_sphere][0], //x2
        (*current_sphere)[1], //y1
        (*neighbor)[j_sphere][1]) //y2
    )
        current_sphere = current_cell->erase(current_sphere);
    else
        current_sphere++;
    }
}
}

```

The `checkDistanceConstraints(...)` function checks if the distance between the coordinates of the nanoparticle is less than the minimum distance defined in the constraints, which is equal to `radius * 2.1`, ie such that the nanoparticles do not touch each other. If this constraints are violated, one of the nanoparticles is removed. The final distribution pattern is obtained as a result of this phase. It should be emphasized that the constraints described were chosen for the simulation of this system under study. But as already mentioned, they can be modified or integrated with others, so that you can customize the distribution to be simulated. The result of this first phase can be seen in the following figure:

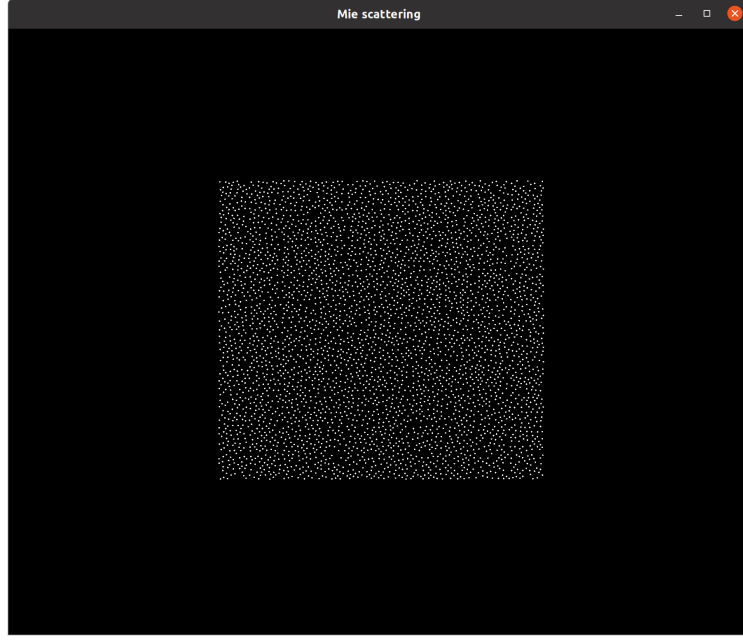


FIGURE 3.5: How is the distribution of nanoparticles for an area of $3\mu m^2$ and a total of 4.072 nanoparticles survivors, after the application of constraints.

3.4.3 Simulation phase

Before introducing the simulation phase, it is necessary to underline that everything is based on the wavelengths and extinction curves of the material being analyzed, i.e. the material of which the nanoparticles are made, which is defined by means of the following arrays:

```
double *wave_lengths;
complex<double> *real_and_imaginary_parts;
UtilsInput::init_wave_lengths(wave_lengths);
UtilsInput::init_real_and_imaginary_parts(real_and_imaginary_parts);
```

- wave_lengths, the wavelengths in analysis expressed in nm.
- real_and_imaginary_parts, i.e. real and imaginary part of the complex refractive index ($\tilde{n} = n - ik$) constituted respectively by the real part n and

the imaginary part k , represent the extinction curves of the material under analysis.

The system takes this information as input in order to be able to calculate the dielectric constant of the material (ϵ) expressed as $(n - ik)^2$, and gives way to the analysis of the spectroscopic behavior of the nanoparticles, simulating an electromagnetic wave that affects a determined distribution of nanoparticles, which as previously described have been stored inside the cells in which they fall. Furthermore, as regards the electromagnetic wave, it can have two polarizations, a Transverse Magnetic polarization (TM) or a Transverse Electric polarization (TE), which can be easily selected with the aid of a Boolean variable specifically defined for this purpose. In the tests carried out during this thesis work, in order to understand the effect of plasmonic coupling of random distributions, the use of TM polarization was chosen. Once the simulation has started, the system processes the response obtained from the distribution of nanoparticles under examination for each wavelength and refractive index of the material, thus carrying out a spectroscopic analysis of the distribution of nanoparticles. As previously mentioned, this system response is defined in terms of absorption, diffusion and extinction, respectively in the variables **Qabs**, **Qsca** and **Qext**. In physical terms, these are energy flows, which quantify the energy transferred by the electromagnetic wave on the nanoparticles, which, interacting with the defined material, determines, based on the wavelength value itself and the current refraction index a different response of the system in terms of energy, in fact there will be areas in which there will be more absorption and others in which instead there will be greater diffusion. These absorption and diffusion values are respectively quantified in terms of **Qabs** and **Qsca** variables. **Qabs** therefore determines the flow of energy absorbed by each nanoparticle and **Qsca** determines the flow of energy scattered of each nanoparticle, for the latter are used detectors defined in the code in the form of arrays called **S1** and **S2**, these detectors can be imagined as a box that covers the nanoparticles which allows to detect the quantity of light of the nanoparticle that crosses the box itself and then calculate the **Qsca** scattered component. Finally we have **Qext**, that is the extinction which is the result of the sum between the absorption component and the scattered component. All these components are finally added together in accordance with physics to estimate the overall quantities of the entire distribution. The simulation phase is implemented through the `gmie(...)` function, in which the distribution of nanoparticles and all the other

data required in order to calculate the energy flows described above are called up. In this method the nanoparticles are identified through their cells of belonging and as already described in the paragraph dedicated to the neighborhood, each nanoparticle of a cell is compared with all the other nanoparticles belonging to the same cell and with those in neighboring cells, according to the scheme of the neighborhood itself. Then the flows will be calculated for each nanoparticle and added at the end to obtain the flows of the distribution as a whole. In order to implement all this, we have chosen to divide the type of operations necessary into two distinct cases, implemented in separate functions, the first function called `single_particle(...)` and the second call `double_particle(...)`, respectively, a function that considers one nanoparticle at a time and another that considers two at a time. Both are called by the function `gmie(...)`, the choice of which of the two to call is decided based on the distance between the current nanoparticle and its neighbors. In fact, a minimum distance has been defined within which the possible neighboring nanoparticle must also be taken into consideration during the calculation phase and therefore quantify the influence that the latter exerts on that current in terms of energy. When a neighboring nanoparticle falls within this range, the choice falls on the `double_particle(...)` function, so this function will be called for each nanoparticle close to the current nanoparticle for which this circumstance occurs. If, on the other hand, none of the neighboring nanoparticles is close enough to be included in the case described above, the flows of the nanoparticle will be calculated exclusively on itself, thus avoiding taking into consideration the nanoparticles belonging to the same neighborhood which they are not close enough to influence the energy of the current nanoparticle. The `single_particle(...)` function will calculate the flows of Q_{abs} , Q_{sca} and Q_{ext} simply by considering the incidence of the electromagnetic wave on the material of the single nanoparticle, while in the `double_particle(...)` function as they will be taken into account two nanoparticles at a time, the distance between them will be calculated always taking into account the chosen polarization and this distance will then be used in the plasmon rule equation, in order to quantify the influence that the neighboring nanoparticle exerts on that current and finally consider the incidence of the electromagnetic wave on the material of both nanoparticles in order to calculate the resulting energy values. The code just described is represented in the appendix [A](#). Once the code just described has been completed, and after making sure that the results of the system are correct, some changes have been made in order to further optimize the execution of the system, to this end

the approach of the system has been slightly modified in the simulation phase. As shown in the `gmie(...)` function, whenever the system response for a given wavelength and a given complex refractive index is simulated, the function performs the comparison operations in order to determine the distances between the nanoparticles and then decide whether to perform the flow calculation with the `single_particle(...)` or `double_particle(...)` function. To overcome this, a different approach has been adopted, that is, since the distribution of the nanoparticles does not change over time, consequently each nanoparticle will always be evaluated with the same neighboring nanoparticles if these are close enough to each other, while it will always be considered alone if there is no one else close enough to influence it. Therefore, in order to optimize the simulation of the system, a single initial comparison operation was performed, already initially sorting each nanoparticle connected to the respective neighboring nanoparticles that influence it in appropriate logical support structures in order to be able to evaluate them together in the calculation, also moving those that must be taken into consideration individually in a structure dedicated to them. This avoids having to repeat the same comparisons between nanoparticles over and over for the calculation of distances. This new version of the `gmie(...)` function takes these special structures as input, and starts to recall the relative associated function for each nanoparticle in these structures, this avoids the repetition of the distance checks at each evaluation of a new wavelength and complex refractive index. From the various tests performed, the new version of the function made the system on average at least 4.5 times faster than its first version. Details of the new version of the `gmie(...)` function are shown in the appendix [B](#).

3.5 Parallelization

Once the whole system with its relative phases has been implemented and optimized, the transformation of the code from serial to parallel is finally achieved. As already mentioned in the chapter dedicated to parallel computing, by analyzing the structure of the code, both in terms of the logical structures used and the type of algorithm implemented in the simulation phase and not least the very nature of the system to be simulated, it was decided to exploit OpenMP for code parallelization. This is because, not only does the OpenMP approach naturally adapt both to the logical structures used and to the communication model based on shared

memory, but it is also necessary to take into account the portability of the code, in fact with OpenMP the parallel code is identical to the serial one. with the addition of some directives that instruct the compiler on how to process the data and on which portions of the same the threads in the parallel version must work. These directives will then simply be ignored when the code is run on systems that do not support OpenMP, still allowing serial execution of the program. As for the parts of the code to be parallelized, by thoroughly analyzing the implementation of the system, it can be easily seen that the simulation is based on the calculation of the flows for each nanoparticle in each cell, regardless of whether these are then calculated by themselves or are considered together with the neighbors, the results of each one, however, will then be added together, in order to obtain the values of the whole system in its entirety. Therefore the code could be parallelized in several points, listed below:

- parallelization on various wavelengths and refractive index
 - parallelization at the cell level for each wavelength and refractive index
 - parallelization at the nanoparticle level of each cell
- In the first case the parallelization occurs upstream, i.e. for each wavelength of the spectrum and refractive index, the workload is divided between the threads in such a way that each of them works on a wavelength and refractive index different. Then each thread simulates the entire distribution for that input data. The only precautions to be taken into consideration in this case are the addition of support structures in which to store the results of each distribution before resimulating the distribution on other wavelengths and refractive indices. In this regard, arrays have been added in which each thread can save the results obtained, specifically the result of the nanoparticle distribution of the j -th wavelength and refractive index will be saved in the j -th positions of the respective arrays. For this case there are no other precautions, as the variables that require changes in writing by the threads during the calculation phase are implicitly made private by OpenMP, as they are defined within the parallel region, while as regards shared variables they are used only for reading. Therefore there are no problems regarding any loss of information due to overwriting.

- In the second case the parallelization occurs at the cell level, therefore already within the function `gmie(...)`, here the computation load is divided at the cell level, i.e. the threads work in parallel on different cells of the same wavelength and refractive index. In this case, the various energy flows calculated by adding those of each nanoparticle belonging each cell must be added at the end to those belonging to other cells calculated by the other threads, to obtain the final flows of the entire simulation. In this case, some variables used by the various threads, which are defined outside the parallel region must be explicitly indicated as private, in order to create a copy for each thread and appropriately initialized accordingly, so avoiding problems regarding any loss of information due to overwriting. Furthermore, the way in which the energy flows of each cell must be added to those calculated by the other threads on the other cells must also be defined. This in OpenMP can be done by adding a critical region marked with the appropriate directive, in which perform such operations, or by declaring a reduction operation on the variables subject to sum at the end, where the system will deal with these operations in accordance with the type of reduction directive. The second is preferable in terms of performance compared to the first, because distinct partial sums can be performed by the system in parallel anyway, while as regards the critical region, it can only be performed by one thread at a time.
- In the latter case, parallelization occurs at the cell level, i.e. here the workload is divided at the level of nanoparticles belonging to the same cell. In this case the flows of each nanoparticle must finally be added to those calculated by the other threads on the other nanoparticles belonging to the same cell. The precautions to be taken into consideration in this last case are the same as those of the second case, but increase in number, as we enter more deeply into the calculation phase. Because here, there are many more variables defined outside the parallel region, whose purpose needs to be defined via the OpenMP directives.

Of the three cases, the last one was discarded regardless, because there is an increase in the complexity of managing the variables and the appropriate operations to coordinate in order to obtain the correct results, moreover it does not add improvements in performance compared to the others, but on the contrary, there is a degradation, since, parallelizing at this level of depth implies that the resulting

computation parts assigned to each thread are too small, and in this case the cost spent in the small number of operations necessary for thread coordination exceeds the gain resulting from the parallelization itself. Furthermore, any changes to the calculation functions would result in a consequent distortion of the parallelization directives. As for the first two cases, on the other hand, both were applied and studied in this phase, but the portions of the parallelization load assigned to each thread turned out to be very similar in the two cases, in fact this was found in the final results obtained in terms performance of the two versions, which proved to be equivalent. Therefore, in the end, it was preferred to opt for the first case. As it allows a division of paralleling at a higher level that not only proves to be simpler, but above all, any changes to the calculation code or to the cell structure itself in the future will not lead to as many changes to the parallelization directives through any additions / removals of critical regions or reduction operations. More precisely, this is due since here the workload is divided according to the wavelength and refractive index to be studied. The code is shown below:

```
vector<vector<double> *> singleParticles;
map<vector<double> *, vector<vector<double> *>> doubleParticles;
vector<vector<double> *> keys;
Utils::check_single_double(singleParticles, doubleParticles, keys, pol);

#pragma omp parallel for schedule(guided)
for (int j = 0; j < NUM_LINE; j++){

    double wl_0{wave_lengths[j]};
    complex<double> eper_p{real_and_imaginary_parts[j]};

    double Qsca = 0.0;
    double Qabs = 0.0;
    double Qext_single = 0.0;
    double Qext_double = 0.0;
    double Qext_all = 0.0;

    vector<complex<double>> S1(nang, {0.0 + 0.0i});
    vector<complex<double>> S2(nang, {0.0 + 0.0i});
```

```

tie(Qsca, Qabs, Qext_single, Qext_double, S1, S2) =
gmie(singleParticles, doubleParticles, keys, wl_0, nang, eper_p, pol);

    Qext_all = Qext_single + Qext_double;
    Qsca_print[j] = Qsca;
    Qabs_print[j] = Qabs;
    Qext_print_single[j] = Qext_single;
    Qext_print_double[j] = Qext_double;
    Qext_print_all[j] = Qext_all;
    S1_print[j] = S1;
    S2_print[j] = S2;
}

```

Chapter 4

Results and simulations

In this thesis work in order to evaluate the simulation of the implemented system, distributions of gold nanoparticles of which each nanoparticle has the same diameter were considered. Several simulations were carried out, by varying the size of the input distribution, in this way it was possible not only to evaluate how the response in energy terms changed but also how the performance associated with each simulation varied from an IT point of view. To this end, the tests were implemented on a workstation consisting of:

- 2 processor Intel[®] Xeon[®] E5-2650 v3, each of which has 10 physical cores that execute 20 threads, so for a total of 40 threads.
- 16 GB of RAM.

The simulations were carried out with the latest version of the system, which was paralyzed thanks to OpenMP. First of all, the results are illustrated from a physical point of view, that is, how the result varies in energy terms, taking into consideration only the nanoparticles that do not interact with the others, compared to those that interact with others close to them. Subsequently, the sum of the energy values of both are considered so as to better illustrate the variation in the scattering extinction that is obtained when the nanoparticles are influenced by neighboring nanoparticles, thanks to the occurrence of plasmonic coupling. As already mentioned, in order to understand the plasmonic coupling of random distributions, the Transverse Magnetic polarization was used. In fact, in the following graphs the translations of the curves along X will be shown,

a symptom of the consideration of the influence of nearby nanoparticles. The nanoparticles taken into consideration have a radius of 20nm, and as illustrated in the paragraph on the constraints for simulation in the current thesis work, the only constraints applied are, as regards the distribution itself, that each nanoparticle must be at a minimum distance from the others by at least 2.1 times the radius, ie the center-center distance between two nanoparticles must be at least 42nm. In this way the nanoparticles will graze without ever touching each other, thus avoiding nanoparticles stuck together. As already mentioned, all the nanoparticles that do not meet this requirement will be eliminated from the distribution during the creation of the distribution itself. The other constraints used in this simulation concerns the calculation phase, that is, since we have highlighted that in the calculation of the energy flows of each nanoparticle we find two cases in which the first concerns the consideration of a single nanoparticle, while the second considers two nanoparticles at a time, so as to be able to simulate any plasmonic coupling to be taken into consideration in the latter case. The constraints dealing with dividing the first case from the second is also based on the center-center distance between nanoparticles, which must belong to the range between 2.1 times the radius and 3 times the radius. Hence respectively from 42nm to 60nm. This is the range to be considered for a possible plasmonic coupling between two nanoparticles, in this way we avoid considering a possible plasmonic coupling between two nanoparticles that are not close enough to influence each other. Only these constraints have been taken into consideration in the current case study, but as already mentioned it is possible to introduce others to model the distributions at will or study them from another point of view. Below we see the graph representing the extinction of a nanoparticle distribution for an area of $3\mu m^2$ and a total of 4.072 nanoparticles:

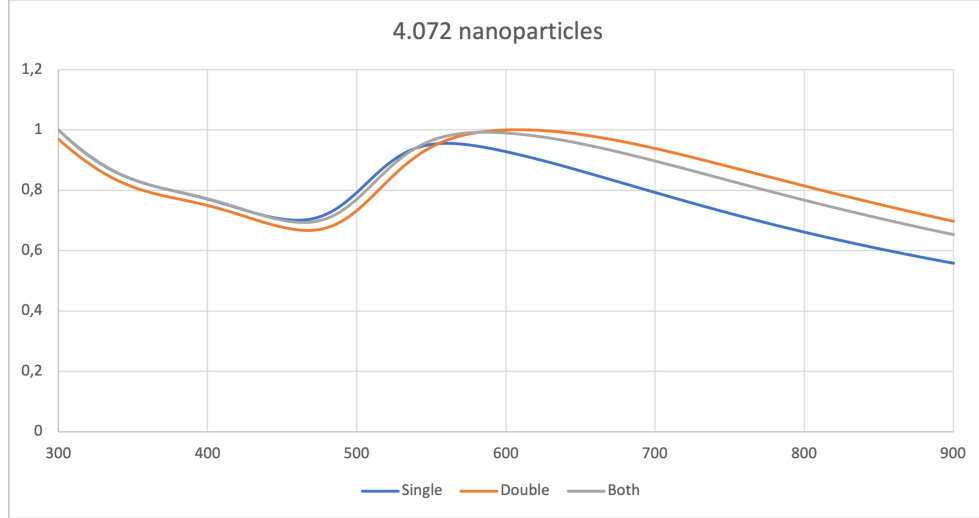


FIGURE 4.1: Extinction curves divided by case for a distribution of 4.072 nanoparticles in an area of $3\mu m^2$.

The number of nanoparticles that meet the requirements of the `double_particle(...)` case are significantly lower than the number of nanoparticles treated by the `single_particle(...)` case. In fact, during the implementation phase of the system it was noted that, due to how the distribution is generated, i.e. not by constraining where the nanoparticles must be generated in space, on average each of them produces a percentage of nanoparticles that falls into the `double_particle(...)` which is around 20%. But despite this, as we can see in the figure, the influence that the consideration of these cases has in energy terms on the entire distribution is considerably perceived. Considering above all, that we are talking about simulating areas of imperceptible dimensions, just to get an idea of the order of magnitude, consider that the diameter of a red blood cell on average is about 8μ .

4.1 Physical results

As for the simulations performed, they were mainly carried out in order to test the response of the system in IT terms, thus considering how the calculation times were extended as the size of the distribution to be studied increased, but below we will also see which is the response of the system from the physical point of view. To this end, the following distributions were considered:

1. area of $3\mu m^2$ with 4.072 nanoparticles
2. area of $5\mu m^2$ with 11.322 nanoparticles
3. area of $8\mu m^2$ with 29.017 nanoparticles
4. area of $10\mu m^2$ with 45.619 nanoparticles
5. area of $15\mu m^2$ with 102.949 nanoparticles
6. area of $20\mu m^2$ with 183.268 nanoparticles
7. area of $25\mu m^2$ with 294.858 nanoparticles
8. area of $30\mu m^2$ with 414.007 nanoparticles
9. area of $35\mu m^2$ with 565.735 nanoparticles
10. area of $40\mu m^2$ with 734.484 nanoparticles
11. area of $45\mu m^2$ with 903.909 nanoparticles
12. area of $50\mu m^2$ with 1.124.322 nanoparticles

The graphs related to distributions 1, 6 and 12 are shown below to show how the absorption energy flow varies in quantitative terms as the size of the distribution increases.

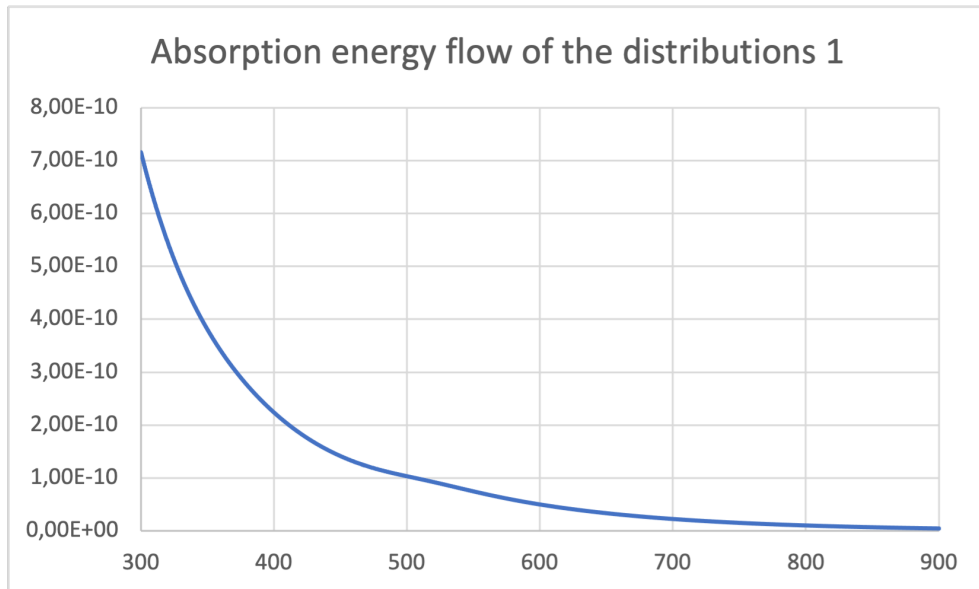


FIGURE 4.2: Absorption curve for a distribution of 4.072 nanoparticles in an area of $3\mu m^2$.

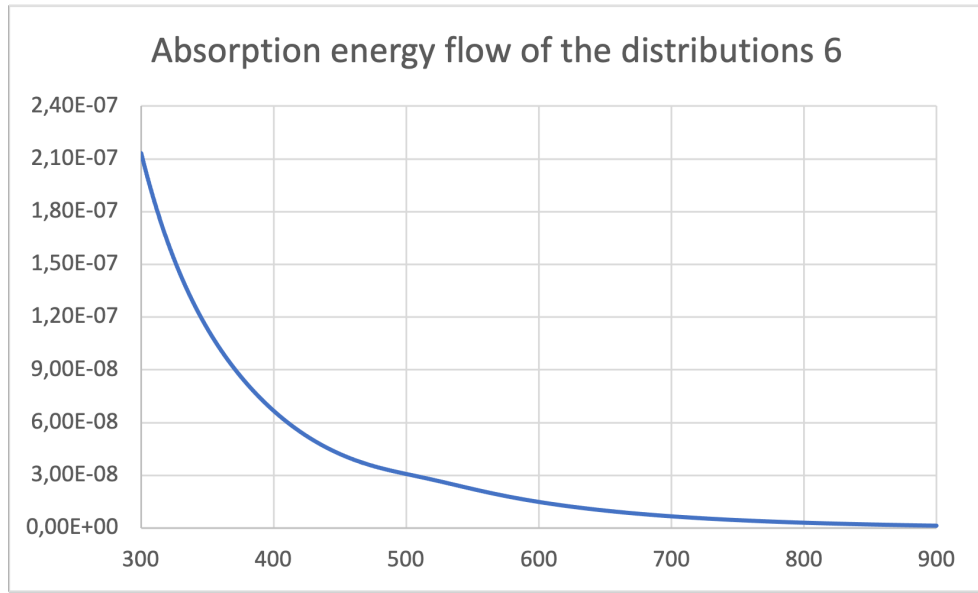


FIGURE 4.3: Absorption curve for a distribution of 183.268 nanoparticles in an area of $20\mu m^2$.

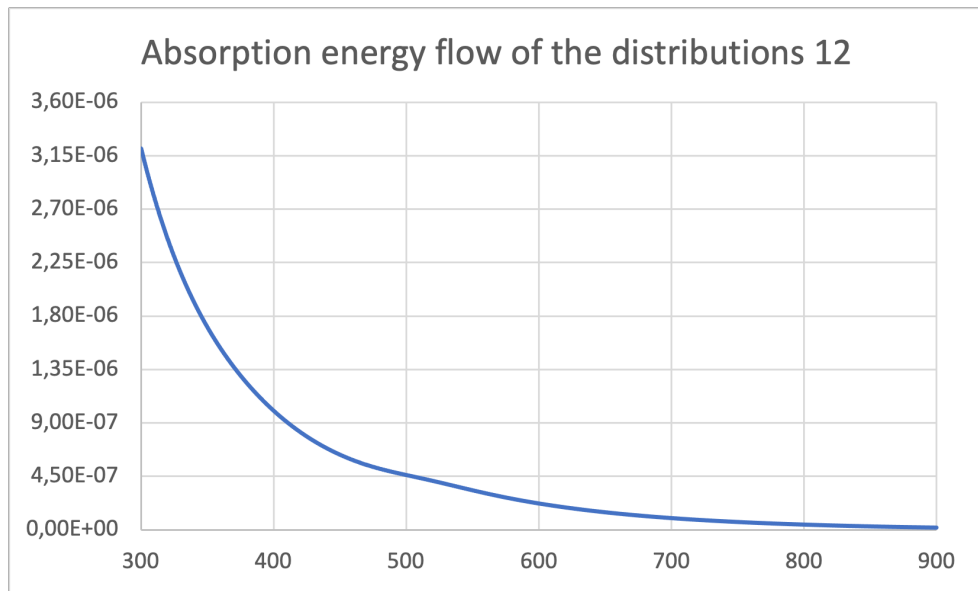


FIGURE 4.4: Absorption curve for a distribution of 1.124.322 nanoparticles in an area of $50\mu m^2$.

It is possible to notice how the absorption curve is reproduced on higher orders of magnitude due to the greater absorption of the nanoparticles.

4.2 Performance results

The previously described distributions have been simulated in the parallel version using two types of worksharing schedules implemented by OpenMP, the first is the static schedule, which takes as input a **CHUNK**, i.e. a dimension, so as to be able to divide the number of for loops among the various threads in portions equal in size to those defined in the **CHUNK**. This allows the programmer to decide how to divide the workload between the various threads. The other schedule used is the guided schedule, which leaves OpenMP the right to choose at execution time which portion to assign to each thread and in what size. In fact, this schedule initially assigns a **CHUNK** of larger size, which during the simulation is gradually resized if the load is poorly balanced in order to approach an equal distribution of the workload between the various threads. Below we see the results obtained from the point of view of system performance using the static schedule.

Times

Threads	$3\mu m^2$	$5\mu m^2$	$8\mu m^2$	$10\mu m^2$	$15\mu m^2$	$20\mu m^2$
1	6,66s	27s	93,66s	210,66s	671,66s	1528,66s
2	3,33s	14s	47,66s	109,66s	347s	801s
4	1,66s	7s	24,66s	56,33s	185,66s	437,33s
8	1s	3,66s	13,66s	30,66s	104s	254,33s
16	0,66s	3s	11s	25,33s	88s	217,66s
32	0,33s	2,33s	8,33s	18s	69,66s	178,33s

Threads	$25\mu m^2$	$30\mu m^2$	$35\mu m^2$	$40\mu m^2$	$45\mu m^2$	$50\mu m^2$
1	3045s	5159,33s	9219,66s	13426,33s	18492,66s	24041s
2	1617s	2760s	4979,33s	7280,66s	10173,33s	13543,33s
4	904,33s	1570s	2851s	4199s	6004,33s	8235,33s
8	547,33s	978s	1786,33s	2671s	3952,33s	5594,66s
16	473,66s	857s	1575s	2345s	3532,33s	5078,66s
32	385,33s	702,66s	1302,33s	1907s	2961s	4382,33s

The previous tables summarize the execution times of the system with the static schedule as the size of the distribution of nanoparticles and the number of threads used vary. While the following tables show the corresponding speedup results, for

the same type of schedule.

Speedup

Threads	$3\mu m^2$	$5\mu m^2$	$8\mu m^2$	$10\mu m^2$	$15\mu m^2$	$20\mu m^2$
1	1,00x	1,00x	1,00x	1,00x	1,00x	1,00x
2	2,00x	1,93x	1,97x	1,92x	1,94x	1,91x
4	4,00x	3,86x	3,80x	3,74x	3,62x	3,50x
8	6,66x	7,38x	6,86x	6,87x	6,46x	6,01x
16	10,09x	9,00x	8,51x	8,32x	7,63x	7,02x
32	20,18x	11,59x	11,24x	11,70x	9,64x	8,57x

Threads	$25\mu m^2$	$30\mu m^2$	$35\mu m^2$	$40\mu m^2$	$45\mu m^2$	$50\mu m^2$
1	1,00x	1,00x	1,00x	1,00x	1,00x	1,00x
2	1,88x	1,87x	1,85x	1,84x	1,82x	1,78x
4	3,37x	3,29x	3,23x	3,20x	3,08x	2,92x
8	5,56x	5,28x	5,16x	5,03x	4,68x	4,30x
16	6,43x	6,02x	5,85x	5,73x	5,24x	4,73x
32	7,90x	7,34x	7,08x	7,04x	6,25x	5,49x

The graphs related to distributions 1, 6 and 12 are shown below to show how speedup curve varies as the size of the distribution increases using the static schedule.

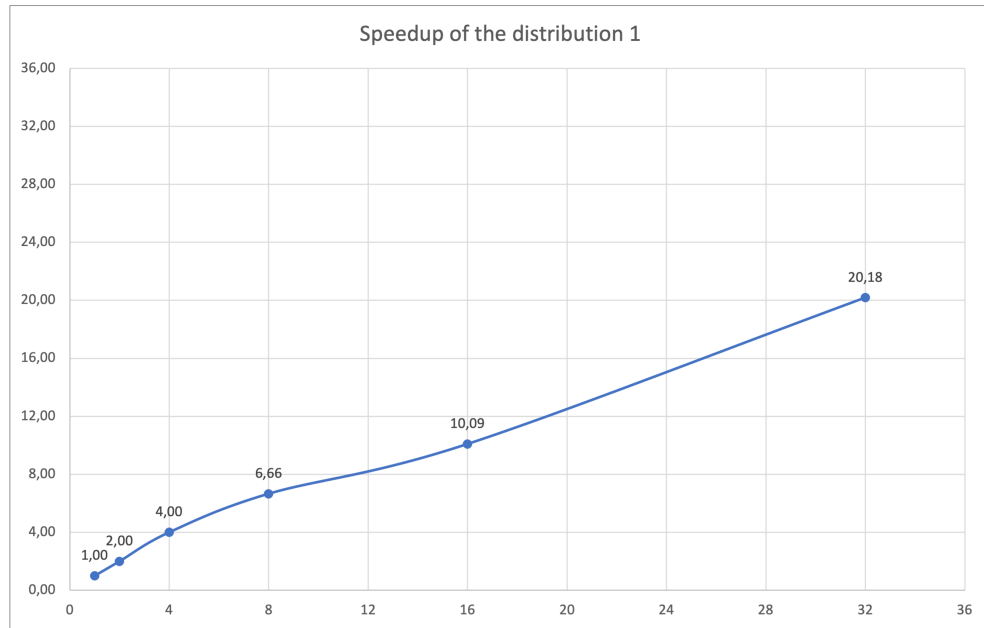


FIGURE 4.5: Speedup curve for a distribution of 4.072 nanoparticles in an area of $3\mu m^2$ with static schedule.

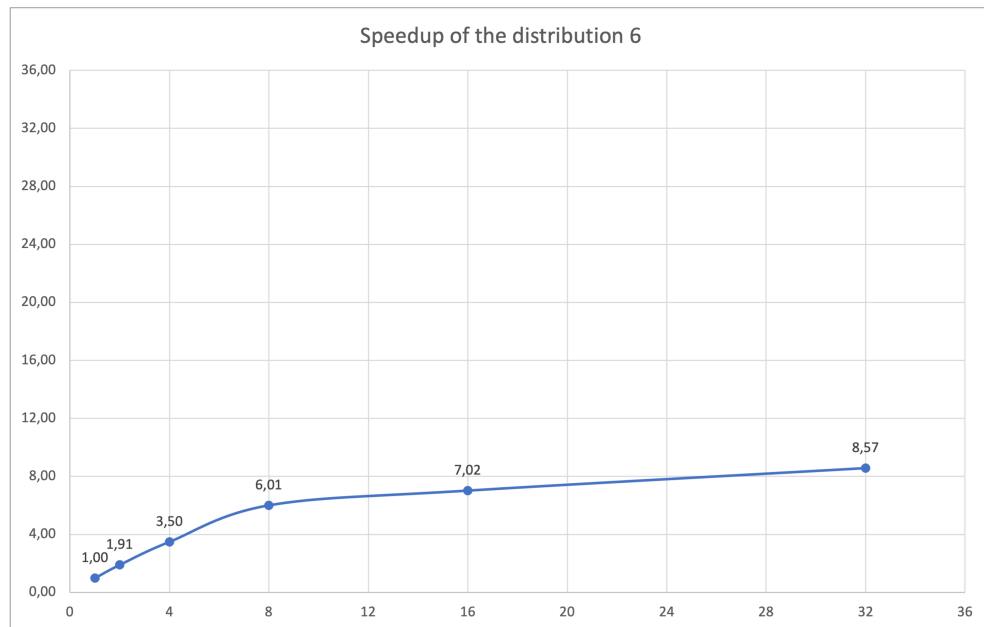


FIGURE 4.6: Speedup curve for a distribution of 183.268 nanoparticles in an area of $20\mu m^2$ with static schedule.

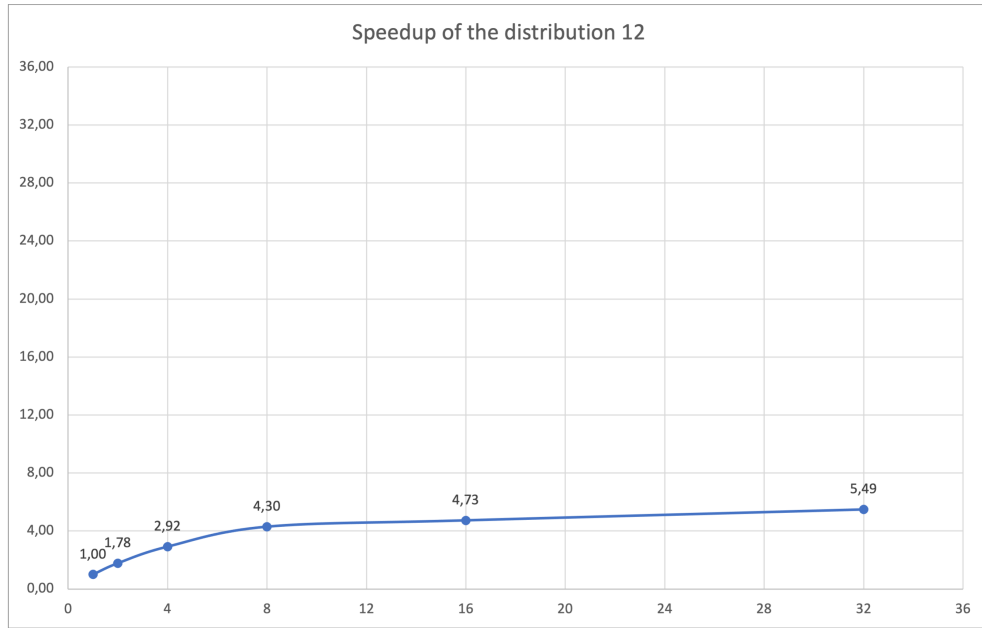


FIGURE 4.7: Speedup curve for a distribution of 1.124.322 nanoparticles in an area of $50\mu m^2$ with static schedule.

The results obtained from the point of view of system performance using the schedule guided are shown below.

Times

Threads	$3\mu m^2$	$5\mu m^2$	$8\mu m^2$	$10\mu m^2$	$15\mu m^2$	$20\mu m^2$
1	6,66s	26,66s	93s	209,66s	670,66s	1527s
2	3,33s	13,66s	47,33s	107s	346,33s	798,66s
4	1,66s	7s	24,33s	55,66s	184,66s	435,33s
8	1s	3,66s	13,33s	30s	103s	253s
16	0,66s	2s	7s	16,33s	61s	158s
32	0,33s	1,66s	6s	13,66s	52s	138s

Threads	$25\mu m^2$	$30\mu m^2$	$35\mu m^2$	$40\mu m^2$	$45\mu m^2$	$50\mu m^2$
1	3031s	5134,33s	9218,33s	13413s	18495,33s	24012s
2	1609,33s	2758,33s	4971,66s	7272,33s	10180,33s	13518s
4	900,33s	1569s	2853,33s	4202s	6035,66s	8258,66s
8	545,33s	976s	1790,33s	2660,33s	3937s	5552,66s
16	360,33s	668,33s	1235,33s	1859s	2865s	4157s
32	323,66s	619s	1142s	1734,66s	2697s	3969,66s

The previous tables summarize the execution times of the system with the guided schedule as the size of the distribution of nanoparticles and the number of threads used vary. While the following tables show the corresponding speedup results, for the same type of schedule.

Speedup

Threads	$3\mu m^2$	$5\mu m^2$	$8\mu m^2$	$10\mu m^2$	$15\mu m^2$	$20\mu m^2$
1	1,00x	1,00x	1,00x	1,00x	1,00x	1,00x
2	2,00x	1,95x	1,96x	1,96x	1,94x	1,91x
4	4,00x	3,81x	3,82x	3,77x	3,63x	3,51x
8	6,66x	7,28x	6,98x	6,99x	6,51x	6,04x
16	10,09x	13,33x	13,29x	12,84x	10,99x	9,66x
32	20,18x	16,06x	15,50x	15,35x	12,90x	11,07x

Threads	$25\mu m^2$	$30\mu m^2$	$35\mu m^2$	$40\mu m^2$	$45\mu m^2$	$50\mu m^2$
1	1,00x	1,00x	1,00x	1,00x	1,00x	1,00x
2	1,88x	1,86x	1,85x	1,84x	1,82x	1,78x
4	3,37x	3,27x	3,23x	3,19x	3,06x	2,91x
8	5,56x	5,26x	5,15x	5,04x	4,70x	4,32x
16	8,41x	7,68x	7,46x	7,22x	6,46x	5,78x
32	9,36x	8,29x	8,07x	7,73x	6,86x	6,05x

The graphs related to distributions 1, 6 and 12 are shown below to show how speedup curve varies as the size of the distribution increases using the guided schedule.

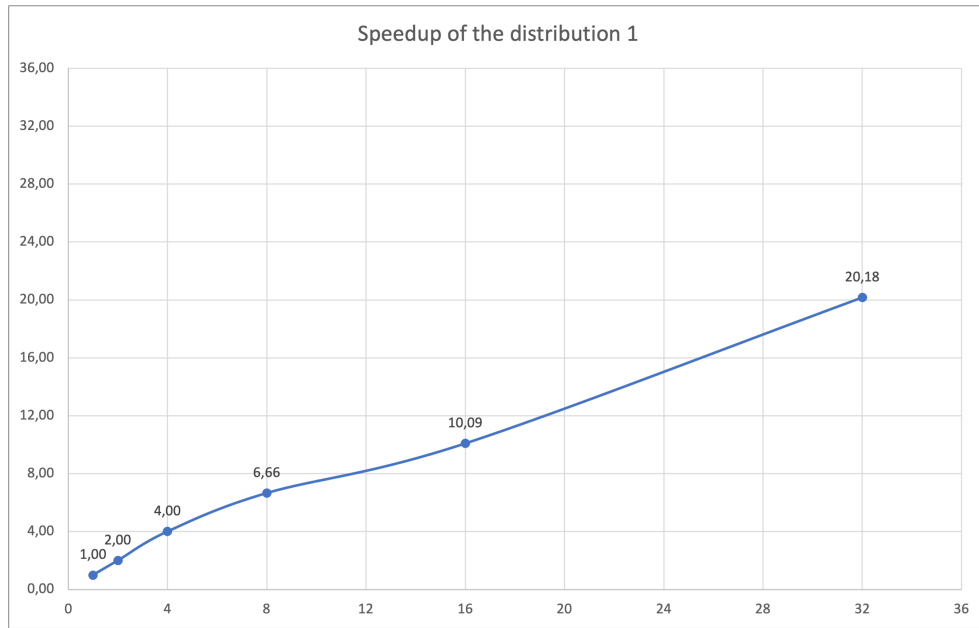


FIGURE 4.8: Speedup curve for a distribution of 4.072 nanoparticles in an area of $3\mu m^2$ with guided schedule.

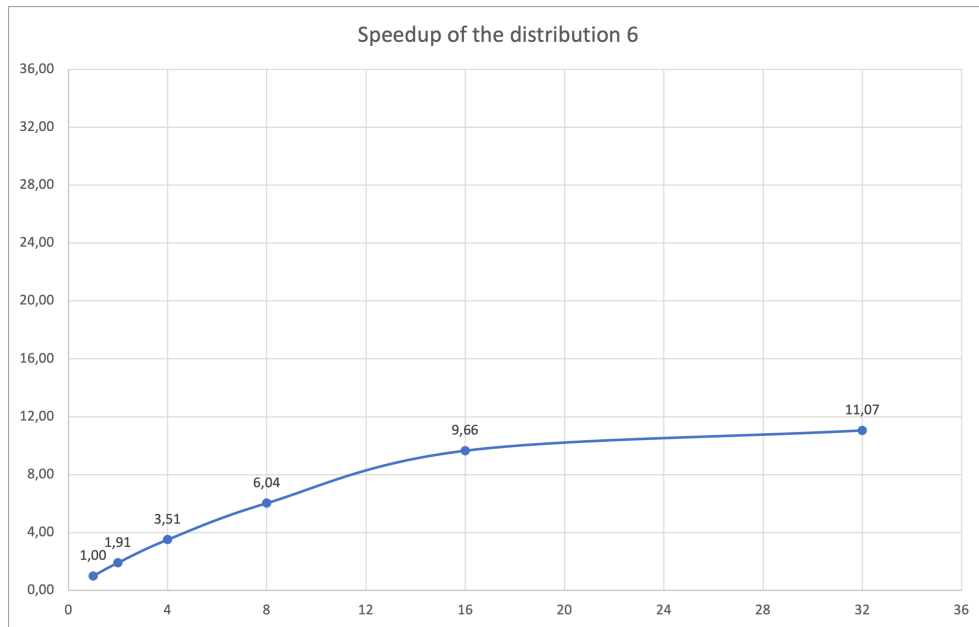


FIGURE 4.9: Speedup curve for a distribution of 183.268 nanoparticles in an area of $20\mu m^2$ with guided schedule.

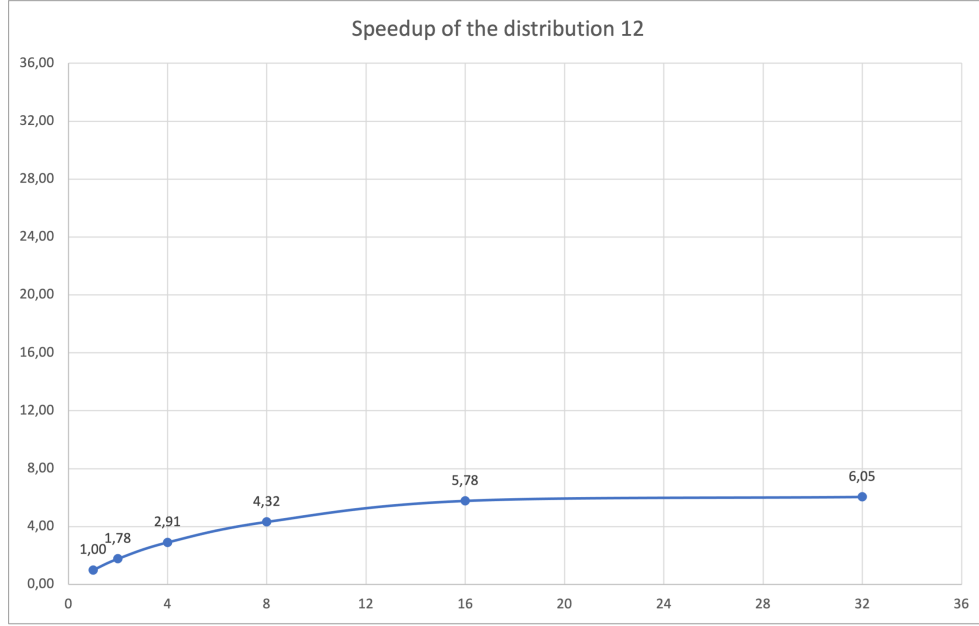


FIGURE 4.10: Speedup curve for a distribution of 1.124.322 nanoparticles in an area of $50\mu m^2$ with guided schedule.

4.3 Performance evaluations

First of all, it must be emphasized that among the speedup curves of each distribution, the curve relating to distribution 1, with both types of schedules, does not seem to follow the trend of the other curves. In fact, only for this case, the resulting curve seems more similar to a straight line, which represents the unusual case of an increase in linear performance as the threads increase. In reality, the reasons for which such a trend could occur may not be due to system performance, but could instead be due to hardware reasons. What happens in these cases from the hardware point of view is very simple, that is, the data used to solve the problem, despite being partitioned into smaller portions each relative to its thread, these portions normally reside in RAM, therefore the various threads during the computation they require access to it to fetch the data they need, making this a limiting factor for performance, but when the data used for solving the problem is quite small, and these being further divided among the various threads, give life to portions of data so small that they can all be stored in their respective caches. Therefore, during the computation phase, the threads will not need to

request data from RAM, but will be able to fetch it directly from their respective cache, so consequently all this will easily be reflected in a significant increase in performance. From this it follows that the resulting speedup curve of distribution 1, at the final evaluation point, could be the result of a hardware circumstance, in fact this result occurred only in the presence of these conditions, i.e. with a very small nanoparticle distribution and a high number of threads. Precisely for being the only isolated case, in which these characteristics are present, it suggests an apparently better result due to hardware circumstances that occur in such situations. In these cases, if you had the possibility to run the test on a machine composed of an even greater number of cores, for the case in question at least 64, it is highly likely that you will get even better results than these, and still increasing the number of cores up to 128 or 256 could even exceed the P speedup limit itself, where P equals the number of cores used. Speedup greater than P is possible only if each process (processor) takes less than T_s/P time in solving the problem, where T_s is the serial time obtaining the so called **superlinear speedup**. As reported in the tables, there is a better response of the system in the use of the guided schedule, surely because by adapting the size of the CHUNK during the execution it is able to better balance the workload between the various threads, even if these improvements seem to disappear as the size of the distribution increases.

To our knowledge, there are no other system implementations that approach the resolution of the current problem, or in any case nothing based on programming languages such as C/C++ has been found, therefore there is no direct comparison with similar systems to have a term of comparison. The only noticed trend concerns the conspicuous loss of speedup as the size of the simulated distribution increases. Usually the opposite would be expected, that is, by simulating a greater distribution, an improvement in speedup would be obtained up to a certain threshold, after which at most it would remain constant, but would not get worse. Usually, however, when a deterioration occurs, this is associated with onerous computation phases that not only limit the computation itself, but even slows it down when their execution becomes more consistent, nullifying the improvements obtained in other phases of the computation. To find out where the cause of all this is, the system execution was analyzed using the VTune Profiler software, which is an Intel-owned performance analysis tool. It allows to analyze the execution of the system in detail, in order to extrapolate information about the execution of the functions called in the code. This allows to understand how the work is divided on individual processors, and how long those functions are

performed. From the analysis carried out, the VTune Profile software highlighted which were the hotspots of the program, that is the most active functions in the execution of the program.

Top Hotspots

This section lists the most active functions in your application. Optimizing these hotspot functions typically results in improving overall application performance.

Function	Module	CPU Time [®]
<code>double_particle</code>	<code>nano_particles</code>	119.236s
<code>__divdc3</code>	<code>libgcc_s.so.1</code>	88.092s
<code>Utils::generatePattern._omp_fn.0</code>	<code>nano_particles</code>	33.642s
<code>single_particle</code>	<code>nano_particles</code>	17.260s
<code>operator new</code>	<code>libstdc++.so.6.0.28</code>	12.609s
[Others]	N/A*	55.716s

*N/A is applied to non-summable metrics.

FIGURE 4.11: Hotspots of the system highlighted by VTune Profiler.

As shown in the figure, among the various hotspots, the function `divdc3(...)` of the `libgcc_so.1` runtime library of the `g++` compiler is evidenced, which implements the division operation between complex numbers. From this we can deduce the reason why a marked loss of speedup was noticed in the tests as the size of the distribution increased. The particularity of this circumstance makes us speculate, because a simple division operation requires computational resources greater than those required for the execution of the main functions of the system. Most likely this function defined in the `libgcc_so.1` runtime library has thread-safe features, which limit its execution in parallel. This came out more with the increase in the size of the distribution, because consequently there is also a greater number of nanoparticles that are simulated by the `double_particle(...)` function, in which more division operations are carried out between complex numbers. As already mentioned, the cases treated by the `double_particle(...)` function are considerably lower in percentage terms than those treated by the `single_particle(...)` function, but despite this, the figure highlights an exaggerated use of computational resources of the `double_particle(...)` function, compared to the `single_particle(...)` function, due precisely to the slowdown caused by the `divdc3(...)` function.

Chapter 5

Conclusions

As already described, the thesis aimed to implement a system that could generate distribution of nanoparticles in a random way, on which to then carry out a study of Mie theory. Therefore, it was necessary to apply the aforementioned theory taking into account more nanoparticles not only in quantitative terms, but above all to take into account the influence that the nanoparticles exert between them when they are within a certain distance, moreover the system also had to allow an easy adding or removing constraints in order to be able to model the study of distribution as desired. One of the main requirements of this system had to be performance, that is, it was necessary to overcome the limitations present on the most common commercial software that use different approaches which consequently fail to take into account more than a few dozen nanoparticles. This objective has been amply satisfied as the system is able in a short time to simulate distributions of more than 1 million nanoparticles, thus managing to recreate the extinction curves resulting from the application of the theory. To fulfill these objectives, a system was therefore created divided into several parts, a first part used for the creation of nanoparticles and their representation in space, a second part used for the application of constraints defined in the code with the aim of modeling such distribution according to the purposes of the study and a last part dedicated to the application of the theory that takes into account the influence to which each nanoparticle is subject when it is in a certain proximity to the others. As already anticipated until today it was difficult to find a code or a program that would allow you to easily simulate distributions of multiple nanoparticles, in fact most of the codes found on the web took into consideration a single nanoparticle or as in the case of COMSOL, through the creation of a single mesh representative

of several nanoparticles it was possible to try to simulate something very similar to a distribution. Obviously, as already described in the appropriate paragraph, this approach reflects negatively from the performance point of view, not allowing to take into account many nanoparticles. The results obtained have shown how the system is able to reproduce the curve for the entire resonance spectrum for each distribution, furthermore it has been shown how distributions of different dimensions respond in energy terms, when they are considered with the same wavelengths and identical type of material. Above all, it has been highlighted how the consideration of the influence of nearby nanoparticles has a drastic impact on the energy values of the single ones that are influenced by these, obtaining greater resonances overall. From the IT point of view, however, it has been highlighted how the system already in its serial version can easily simulate considerable distributions, the addition of the OpenMP library has further emphasized this strength, not only for the performance advantages of parallelization but also for a question of portability. To this end, as discussed in the paragraph dedicated to performance, there are still some problems to be solved in order to take advantage of the maximum efficiency of the system. One of the future objectives is precisely to examine in depth the problem relating to the loss of speedup of the system when it simulates larger distributions. Once this problem has been solved, we could think of an expansion of the system that allows us to study distribution, also considering it in thermal terms. Furthermore, a less relevant, but still important part would be the improvement of the part dedicated to the definition of constraints, which could be made more user friendly, the same applies to the definition of the material.

Appendix A

Simulation phase

```
#include "gmie.h"

const complex<double> eper_m{1.0 + 0.0i};
const complex<double> mper_m{1.0 + 0.0i};
const complex<double> m_m = sqrt(eper_m * mper_m);
const complex<double> mper_p{1.0 + 0.0i};
const double r_p{0.02};

tuple<double, double, double, double, vector<complex<double>>,
vector<complex<double>>> gmie(const double &wl_0,
const int &nang, const complex<double> &eper_p, const bool &pol){

complex<double> m_p = sqrt(eper_p * mper_p);
complex<double> m_r = m_p / m_m;
double k0 = 2 * M_PI / wl_0;
complex<double> k = m_m * k0;

vector<complex<double>> S1(nang, {0.0 + 0.0i}), S2(nang, {0.0 + 0.0i});
double Qsca{0.0}, Qext_single{0.0}, Qext_double{0.0}, Qabs{0.0};
map<int, vector<vector<double>>> &cells = Utils::cells;

for (int key = 0; key < CELLS; key++){
    int numNeighbors;
    int neighbors[NEIGHBORS];
```

```

Utils::getNeighborsForComputation(key, neighbors, numNeighbors);

for (int i = 0; i < cells[key].size(); i++){
    bool single = true;

    //particles in the same cell

    for (int j = i + 1; j < cells[key].size(); j++)
        if (Utils::checkDistanceBetweenParticles(cells[key][i][0],
            cells[key][i][1], cells[key][j][0], cells[key][j][1], pol)){

            double_particle(cells[key][i][0], cells[key][i][1],
                cells[key][j][0], cells[key][j][1], k, m_r, mper_p,
                mper_m, nang, Qsca, Qext_double, Qabs, S1, S2, pol);

            single = false;

        }

    //particles in neighboring cells

    //for each neighbor
    for (int current = 0; current < numNeighbors; current++)
        //for each particle of each neighbor
        for (int j = 0; j < cells[neighbors[current]].size(); j++)
            if (Utils::checkDistanceBetweenParticles(cells[key][i][0],
                cells[key][i][1], cells[neighbors[current]][j][0],
                cells[neighbors[current]][j][1], pol)){

                double_particle(cells[key][i][0], cells[key][i][1],
                    cells[neighbors[current]][j][0], cells[neighbors[current]][j][1],
                    k, m_r, mper_p, mper_m, nang, Qsca, Qext_double, Qabs, S1, S2,
                    pol);

                single = false;
            }
}

```

```

    }
    if (single)
        single_particle(cells[key][i][0], cells[key][i][1], k, m_r,
            mper_p, mper_m, nang, Qsca, Qext_single, Qabs, S1, S2);
    }

}

return forward_as_tuple(Qsca, Qabs, Qext_single, Qext_double, S1,
    S2);
}

void single_particle(const double &xPos, const double &yPos,
    const complex<double> &k, const complex<double> &m_r,
    const complex<double> &mper_p, const complex<double> &mper_m,
    const int &nang, double &QscaTmp, double &QextTmp, double &QabsTmp,
    vector<complex<double>> &S1Tmp, vector<complex<double>> &S2Tmp){

    complex<double> x = xPos * k * r_p;
    complex<double> y = yPos * k * m_r * r_p;

    int nstop = floor(abs(x) + 4 * cbrt(abs(x)) + 2);
    int nmx = max(nstop, static_cast<int>(ceil(abs(y)))) + 15;

    vector<complex<double>> DD(nmx, {0.0 + 0.0i});
    for (int n = nmx - 1; n > 1; -n)
        DD[n - 2] = (1.0 * n) / y - 1.0 / (DD[n - 1] + (1.0 * n) / y);

    vector<complex<double>> RR(nmx + 1, {0.0 + 0.0i});
    RR[nmx] = x / (2.0 * nmx + 1.0);
    for (int n = nmx - 1; n > -1; -n)
        RR[n] = 1.0 / ((2.0 * n + 1.0) / x - RR[n + 1]);

    vector<complex<double>> psi(nmx + 1, {0.0 + 0.0i});
    psi[0] = RR[0] * cos(x);
    for (int n = 1; n <= nstop; ++n)

```

```

    psi[n] = RR[n] * psi[n - 1];

vector<complex<double>> chi(nstop + 1, {0.0 + 0.0i});
chi[0] = -cos(x);
chi[1] = (1.0 / x) * chi[0] - sin(x);
for (int n = 2; n <= nstop; ++n)
    chi[n] = ((2.0 * n - 1.0) / x) * chi[n - 1] - chi[n - 2];

vector<complex<double>> qsi(nstop + 1, {0.0 + 0.0i});
for (int n = 0; n <= nstop; ++n)
    qsi[n] = psi[n] + chi[n] * complex<double>(0.0 + 1.0i);

fill(RR.begin(), RR.end(), 0.0 + 0.0i);
RR[nmx] = y / (2.0 * nmx + 1.0);
for (int n = nmx - 1; n > -1; --n)
    RR[n] = 1.0 / ((2.0 * n + 1.0) / y - RR[n + 1]);

vector<complex<double>> psim(nstop + 1, {0.0 + 0.0i});
psim[0] = RR[0] * cos(y);
for (int n = 1; n <= nstop; ++n)
    psim[n] = RR[n] * psim[n - 1];

vector<complex<double>> a(nstop, {0.0 + 0.0i}), b(nstop, {0.0 + 0.0i}),
c(nstop, {0.0 + 0.0i}), d(nstop, {0.0 + 0.0i});

for (int n = 0; n < nstop; ++n){

    a[n] = ((mper_p / (mper_m * m_r) * DD[n] + (n + 1.0) / x) *
        psi[n + 1] - psi[n]) / ((mper_p / (mper_m * m_r) * DD[n] +
        (n + 1.0) / x) * qsi[n + 1] - qsi[n]);

    b[n] = ((mper_m * m_r / mper_p * DD[n] + (n + 1.0) / x) *
        psi[n + 1] - psi[n]) / ((mper_m * m_r / mper_p * DD[n] +
        (n + 1.0) / x) * qsi[n + 1] - qsi[n]);

    c[n] = complex<double>(0.0 + 1.0i) * m_r / (psim[n + 1] *

```

```

        (qsi[n] - qsi[n + 1] * (mper_m * m_r / mper_p * DD[n] +
        (n + 1.0) / x)));

d[n] = complex<double>(0.0 + 1.0i) * mper_p / mper_m /
        (psim[n + 1] * (qsi[n] - qsi[n + 1] *
        (mper_p / (mper_m * m_r) * DD[n] + (n + 1.0) / x)));

}

vector<double> fn1(nstop, 0.0), fn2(nstop, 0.0);

for (int n = 0; n < nstop; ++n){
    fn1[n] = 2.0 * (n + 1) + 1.0;
    fn2[n] = fn1[n] / ((n + 1.0) * (n + 2.0));
}

vector<complex<double>> S1(nang, {0.0 + 0.0i}), S2(nang, {0.0 + 0.0i});
vector<double> pie(nstop, {0.0}), tau(nstop, {0.0});

for (int j = 0; j < nang; ++j){
    double dtheta{M_PI / (nang - 1)};
    double theta = j * dtheta;
    double mu = cos(theta);
    pie[0] = 1.0;
    pie[1] = 3 * mu * pie[0];
    tau[0] = mu * pie[0];
    tau[1] = 2 * mu * pie[1] - 3 * pie[0];
    for (int n = 2; n < nstop; ++n){
        pie[n] = ((2.0 * (n + 1.0) - 1.0) / n) * mu * pie[n - 1] -
            ((n + 1.0) / n) * pie[n - 2];
        tau[n] = (n + 1.0) * mu * pie[n] - (n + 2.0) * pie[n - 1];
    }

    for (int n = 0; n < nstop; ++n){
        S1[j] += fn2[n] * (a[n] * pie[n] + b[n] * tau[n]);
        S2[j] += fn2[n] * (a[n] * tau[n] + b[n] * pie[n]);
    }
}

```

```

    }
}

double Qsca{0.0}, Qext{0.0}, Qabs{0.0};

if (imag(x) == 0){

    for (int n = 0; n < nstop; ++n){
        Qsca = Qsca + fn1[n] * (abs(a[n]) * abs(a[n]) + abs(b[n]) *
abs(b[n]));
        Qext = Qext + fn1[n] * real(a[n] + b[n]);
    }

    Qsca *= 2.0 / real(x) / real(x);
    Qext *= 2.0 / real(x) / real(x) * 0.000001;
    Qabs = Qext - Qsca;
}

else{
    Qsca = Qabs = numeric_limits<double>::quiet_NaN();
    Qext = 4.0 / (r_p * r_p * real(k)) *
        imag(S1[0] * complex<double>(0.0 + 1.0i) / real(k));

    for (int i = 0; i < nang; ++i){
        S1[i] = {numeric_limits<double>::quiet_NaN(),
numeric_limits<double>::quiet_NaN()};
        S2[i] = {numeric_limits<double>::quiet_NaN(),
numeric_limits<double>::quiet_NaN()};
    }
}

QscaTmp += Qsca;
QabsTmp += Qabs;
QextTmp += Qext;

for (int i = 0; i < nang; ++i){

```

```

        S1Tmp[i] += S1[i];
        S2Tmp[i] += S2[i];
    }

}

```

```

void double_particle(const double &xPos, const double &yPos,
const double &xPos2, const double &yPos2, const complex<double> &k,
const complex<double> &m_r, const complex<double> &mper_p,
const complex<double> &mper_m, const int &nang, double &QscaTmp,
double &QextTmp, double &QabsTmp, vector<complex<double>> &S1Tmp,
vector<complex<double>> &S2Tmp, const bool &pol){

```

```

    complex<double> x = k * r_p;
    complex<double> y = k * m_r * r_p;

```

```

    double distance = 0.0;

```

```

    if (pol)
        distance = -abs(xPos2 - xPos);
    else
        distance = -abs(yPos2 - yPos);

```

```

    distance = distance - (2 * r_p);
    distance = distance / (2 * r_p);

```

```

    double distancePointToPoint = pow(M_E, (0.18 * (-distance))) / 0.23);

```

```

    int nstop = floor(abs(x) + 4 * cbrt(abs(x)) + 2);
    int nmx = max(nstop, static_cast<int>(ceil(abs(y)))) + 15;

```

```

    vector<complex<double>> DD(nmx, {0.0 + 0.0i});
    for (int n = nmx - 1; n > 1; -n)
        DD[n - 2] = distancePointToPoint * (1.0 * n) /

```

```

        y - 1.0 / (DD[n - 1] + (1.0 * n) / y);

vector<complex<double>> RR(nmx + 1, {0.0 + 0.0i});
RR[nmx] = x / (2.0 * nmx + 1.0);
for (int n = nmx - 1; n > -1; --n)
    RR[n] = 1.0 / ((2.0 * n + 1.0) / x - RR[n + 1]);

vector<complex<double>> psi(nmx + 1, {0.0 + 0.0i});
psi[0] = RR[0] * cos(x);
for (int n = 1; n <= nstop; ++n)
    psi[n] = distancePointToPoint * RR[n] * psi[n - 1];

vector<complex<double>> chi(nstop + 1, {0.0 + 0.0i});
chi[0] = -cos(x);
chi[1] = (1.0 / x) * chi[0] - sin(x);
for (int n = 2; n <= nstop; ++n)
    chi[n] = ((2.0 * n - 1.0) / x) * chi[n - 1] - chi[n - 2];

vector<complex<double>> qsi(nstop + 1, {0.0 + 0.0i});
for (int n = 0; n <= nstop; ++n)
    qsi[n] = psi[n] + chi[n] * complex<double>(0.0 + 1.0i);

fill(RR.begin(), RR.end(), 0.0 + 0.0i);
RR[nmx] = y / (2.0 * nmx + 1.0);
for (int n = nmx - 1; n > -1; --n)
    RR[n] = 1.0 / ((2.0 * n + 1.0) / y - RR[n + 1]);

vector<complex<double>> psim(nstop + 1, {0.0 + 0.0i});
psim[0] = RR[0] * cos(y);
for (int n = 1; n <= nstop; ++n)
    psim[n] = distancePointToPoint * RR[n] * psim[n - 1];

vector<complex<double>> a(nstop, {0.0 + 0.0i}), b(nstop, {0.0 + 0.0i}),
    c(nstop, {0.0 + 0.0i}), d(nstop, {0.0 + 0.0i});

for (int n = 0; n < nstop; ++n){

```

```

a[n] = ((mper_p / (mper_m * m_r) * DD[n] + (n + 1.0) / x) *
        psi[n + 1] - psi[n]) / ((mper_p / (mper_m * m_r) *
        DD[n] + (n + 1.0) / x) * qsi[n + 1] - qsi[n]);
b[n] = ((mper_m * m_r / mper_p * DD[n] + (n + 1.0) / x) *
        psi[n + 1] - psi[n]) / ((mper_m * m_r / mper_p * DD[n] +
        (n + 1.0) / x) * qsi[n + 1] - qsi[n]);
c[n] = complex<double>(0.0 + 1.0i) * m_r / (psim[n + 1] *
        (qsi[n] - qsi[n + 1] * (mper_m * m_r / mper_p * DD[n] +
        (n + 1.0) / x)));
d[n] = complex<double>(0.0 + 1.0i) * mper_p / mper_m /
        (psim[n + 1] * (qsi[n] - qsi[n + 1] * (mper_p /
        (mper_m * m_r) * DD[n] + (n + 1.0) / x)));
}

vector<double> fn1(nstop, 0.0), fn2(nstop, 0.0);
for (int n = 0; n < nstop; ++n){
    fn1[n] = 2.0 * (n + 1) + 1.0;
    fn2[n] = fn1[n] / ((n + 1.0) * (n + 2.0));
}

vector<complex<double>> S1(nang, {0.0 + 0.0i}), S2(nang, {0.0 + 0.0i});
vector<double> pie(nstop, {0.0}), tau(nstop, {0.0});
for (int j = 0; j < nang; ++j){
    double dtheta{M_PI / (nang - 1)};
    double theta = j * dtheta;
    double mu = cos(theta);
    pie[0] = 1.0;
    pie[1] = 3 * mu * pie[0];
    tau[0] = mu * pie[0];
    tau[1] = 2 * mu * pie[1] - 3 * pie[0];
    for (int n = 2; n < nstop; ++n){
        pie[n] = ((2.0 * (n + 1.0) - 1.0) / n) * mu *
            pie[n - 1] - ((n + 1.0) / n) * pie[n - 2];
        tau[n] = (n + 1.0) * mu * pie[n] - (n + 2.0) * pie[n - 1];
    }
    for (int n = 0; n < nstop; ++n){

```

```

        S1[j] += fn2[n] * (a[n] * pie[n] + b[n] * tau[n]);
        S2[j] += fn2[n] * (a[n] * tau[n] + b[n] * pie[n]);
    }
}

double Qsca{0.0}, Qext{0.0}, Qabs{0.0};
if (imag(x) == 0){
    for (int n = 0; n < nstop; ++n){
        Qsca = Qsca + fn1[n] * (abs(a[n]) * abs(a[n]) +
                                abs(b[n]) * abs(b[n]));
        Qext = Qext + fn1[n] * real(a[n] + b[n]);
    }
    Qsca *= 2.0 / real(x) / real(x);
    Qext *= 2.0 / real(x) / real(x) * pow(10, -12);
    Qabs = Qext - Qsca;
}
else{
    Qsca = Qabs = numeric_limits<double>::quiet_NaN();
    Qext = 4.0 / (r_p * r_p * real(k)) *
           imag(S1[0] * complex<double>(0.0 + 1.0i) / real(k));
    for (int i = 0; i < nang; ++i){
        S1[i] = {numeric_limits<double>::quiet_NaN(),
                 numeric_limits<double>::quiet_NaN()};
        S2[i] = {numeric_limits<double>::quiet_NaN(),
                 numeric_limits<double>::quiet_NaN()};
    }
}

QscaTmp += Qsca;
QabsTmp += Qabs;
QextTmp += Qext;

for (int i = 0; i < nang; ++i){
    S1Tmp[i] += S1[i];
    S2Tmp[i] += S2[i];
}

```

}

Appendix B

Gmie function

```
tuple<double, double, double, double, vector<complex<double>>,
vector<complex<double>>> gmie(const vector<vector<double>*>
&singleParticles, map<vector<double>*, vector<vector<double>*>>
&doubleParticles, const vector<vector<double>*> &keys,
const double &wl_0, const int &nang, const complex<double> &eper_p,
const bool &pol){

complex<double> m_p = sqrt(eper_p * mper_p);
complex<double> m_r = m_p / m_m;
double k0 = 2 * M_PI / wl_0;
complex<double> k = m_m * k0;

vector<complex<double>> S1(nang, 0.0 + 0.0i), S2(nang, 0.0 + 0.0i);
double Qsca0.0, Qext_single0.0, Qext_double0.0, Qabs0.0;
map<int, vector<vector<double>>> &cells = Utils::cells;

for(int i=0;i<keys.size();i++)
    for (int j = 0; j < doubleParticles[keys[i]].size(); j++)
        double_particle(keys[i][0][0], keys[i][0][1],
            doubleParticles[keys[i]][j][0][0],
            doubleParticles[keys[i]][j][0][1], k,m_r, mper_p,
            mper_m, nang, Qsca, Qext_double, Qabs, S1, S2, pol);
```

```

for (int i = 0; i < singleParticles.size(); i++)
    single_particle(singleParticles[i][0][0],
        singleParticles[i][0][1], k, m_r, mper_p, mper_m,
        nang, Qsca, Qext_single, Qabs, S1, S2);

return forward_as_tuple(Qsca, Qabs, Qext_single, Qext_double, S1, S2);
}

```

Bibliography

- [1] en.wikipedia.org/wiki/Refractive_index
- [2] en.wikipedia.org/wiki/Relative_permittivity
- [3] en.wikipedia.org/wiki/Permittivity
- [4] Liliana Valente, Studio della risposta termo-plasmonica di uno strato omogeneo di nano- particelle d'oro con diversi ricoprimenti di poli-elettrolita, Università della Calabria, a.a. 2018/2019.
- [5] en.wikipedia.org/wiki/Mie_scattering.
- [6] en.wikipedia.org/wiki/Maxwell%27s_equations
- [7] Ugo Cataldi, Plasmonica attiva in maniera soffice drogata con nanoparticelle d'oro, Università della Calabria, 2013.
- [8] Giuseppe De Salvo, Implementazione di simulazioni tramite “Comsol Multiphysics” per lo studio di risonanze plasmoniche superficiali localizzate dell'oro, Università della Calabria, a.a. 2016/2017.
- [9] www.researchgate.net/figure/Schematic-of-localized-surface-plasmon-resonance-LSPR-Coherent-oscillations-in-the_fig2_300850269.
- [10] www.comsol.it
- [11] en.wikipedia.org/wiki/Finite_element_method

- [12] Giuseppe Emanuele Lio, Giovanna Palermo, Roberto Caputo, and Antonio De Luca: "Opto-mechanical control of flexible plasmonic materials", January 2019
- [13] Giuseppe Emanuele Lio, Giovanna Palermo, Antonio De Luca, and Roberto Caputo: "Tensile control of the thermal flow in plasmonic heaters realized on flexible substrates", December 2019
- [14] Giuseppe Emanuele Lio, Antonio De Luca, Cesare Paolo Umeton, and Roberto Caputo: "Opto-mechanically induced thermoplasmonic response of unclonable flexible tags with hotspot fingerprint", September 2020
- [15] Giovanna Palermo, Ugo Cataldi, Antonio Condello, Roberto Caputo, Thomas Bürgi, Cesare Umeton, and Antonio De Luca, "Flexible thermo-plasmonics: an opto-mechanical control of the heat generated at the nanoscale", Journal Name, Published on 01 August 2018.
- [16] Prashant K. Jain, Wenyu Huang, and Mostafa A. El-Sayed, "On the Universal Scaling Behavior of the Distance Decay of Plasmon Coupling in Metal Nanoparticle Pairs: A Plasmon Ruler Equation", Nano Lett., Vol. 7 , No. 7, 2007, pp. 2081-2083, 2086.
- [17] Rohit Chandra, Leonardo Dagum, Dave Kohr, Dror Maydan, Jeff McDonald, Ramesh Menon: "Parallel Programming in OpenMP", January 2001.
- [18] en.wikipedia.org/wiki/Parallel_programming_model
- [19] Alina Kiessling : "An Introduction to Parallel Programming with OpenMP", University of Edinburgh, April 2009
- [20] Pietro Napoli, Bachelor of Science Degree in Computer Science with title "Heterogeneous parallel programming

with OpenMP 4: A preliminary DEM model implementation", UNICAL, Department of Mathematics and Computer Science. A.A. 2016/2017.

- [21] Amit Amritkar, Danesh Tafti, Rui Liu, Rick Kufrin, Barbara Chapman: "OpenMP parallelism for fluid and fluid-particulate systems", June 2012
- [22] Joel Criado, Marta Garcia-Gasulla, Jesús Labarta, Arghya Chatterjee, Oscar Hernandez, Raül Sirvent and Gonzalo Alvarez: "Optimization of Condensed Matter Physics Application with OpenMP Tasking Model", August 2019
- [23] Amit Amritkar, Surya Deb, Danesh Tafti: "Efficient parallel CFD-DEM simulations using OpenMP", September 2013
- [24] G. Latu, N. Crouseilles, V. Grandgirard, and E. Sonnendrücker: "Gyrokinetic Semi-lagrangian Parallel Simulation Using a Hybrid OpenMP/MPI Programming", September 2007
- [25] github.com/nmoteki/gmie_cpp
- [26] Martina Scaramuzzo, Tesi triennale con titolo "Studio numerico alla nanoscala di sistemi plasmonici fortemente accoppiati", UNICAL, Dipartimento di Fisica, Corso di Studio in Scienza dei Materiali Innovativi e per le Nanotecnologie. A.A. 2018/2019. Da pag 7 a 24.

Ringraziamenti

Prima di tutto ringrazio la mia famiglia, che mi ha sostenuto in questo percorso magistrale. Un doveroso ringraziamento va ai professori Spataro e Caputo che sono stati sempre a disposizione durante questo percorso di tesi e uno speciale ringraziamento va a Giuseppe che con la sua disponibilità, non mi ha mai abbandonato e mi ha sempre guidato in questi mesi di duro lavoro.